

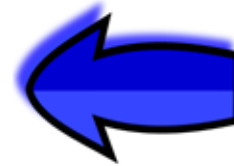


Principles of Pipelining

Basic Computer Architecture

Outline

- * Overview of Pipelining
- * A Pipelined Data Path
- * Pipeline Hazards
- * Pipeline with Interlocks
- * Forwarding
- * Performance Metrics
- * Interrupts/ Exceptions



Up till now

- * We have designed a **processor** that can execute all the **SimpleRisc** Instructions
- * We have look at two **styles** :
 - * With a **hardwired** control unit
 - * **Microprogrammed** control unit
 - * Microprogrammed data path
 - * Microassembly Language
 - * Microinstructions

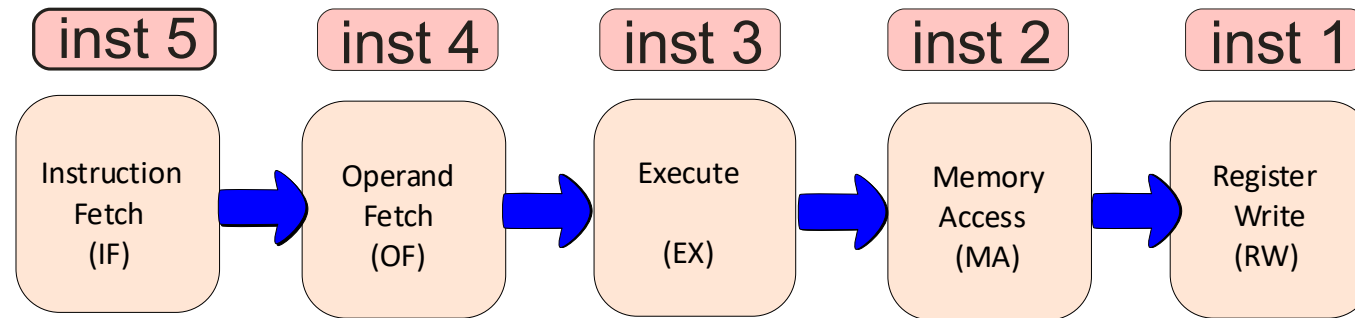
Designing Efficient Processors

- * **Microprogrammed** processors are much slower than **hardwired** processors
- * Even **hardwired** processors
 - * Have a lot of **waste** !!!
 - * We have 5 stages.
 - * What is the IF stage doing, when the MA stage is active ?
 - * **ANSWER** : It is **idling**

The Notion of Pipelining

- * Let us go back to the **car assembly line**
 - * Is the **engine shop** idle, when the **paint shop** is painting a car ?
 - * **NO** : It is building the engine of another car
 - * When this engine goes to the body shop, it builds the engine of another car, and **so on**
 - * **Insight** :
 - * **Multiple cars** are built at the same time.
 - * A car **proceeds** from one stage to the next

Pipelined Processors



- * The IF, ID, EX, MA, and RW stages process 5 instructions simultaneously
- * Each instruction proceeds from one stage to the next
- * This is known as pipelining

Advantages of Pipelining

- * We keep all parts of the data path, busy all the time
- * Let us assume that all the 5 stages do the same amount of **work**
 - * **Without** pipelining, every **T** seconds, an instruction completes its **execution**
 - * **With** pipelining, every **T/5** seconds, a new instruction completes its **execution**

Design of a Pipeline

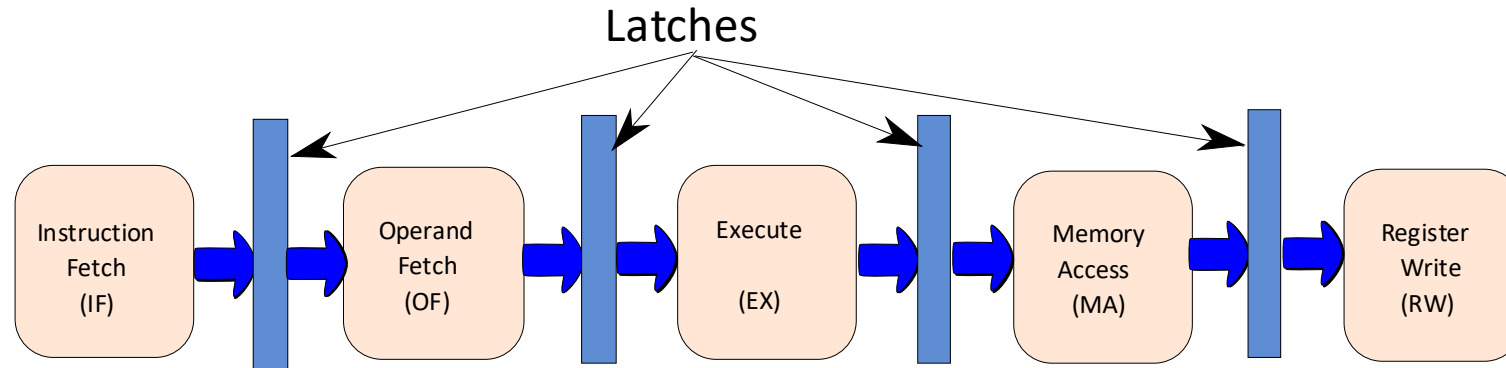
- * Splitting the Data Path

- * We divide the data path into 5 parts : IF, OF, EX, MA, and RW

- * Timing

- * We insert latches (pipeline registers) between consecutive stages
 - * 4 Latches → IF-OF, OF-EX, EX-MA, and MA-RW
 - * At the negative edge of a clock, an instruction moves from one stage to the next

Pipelined Data Path with Latches



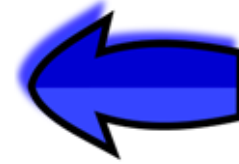
- * Add a latch between subsequent stages.
 - * Triggered by a negative clock edge

The Instruction Packet

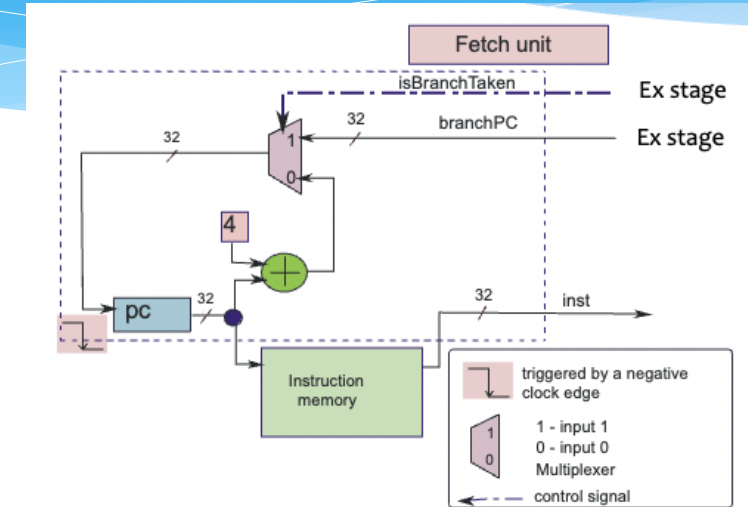
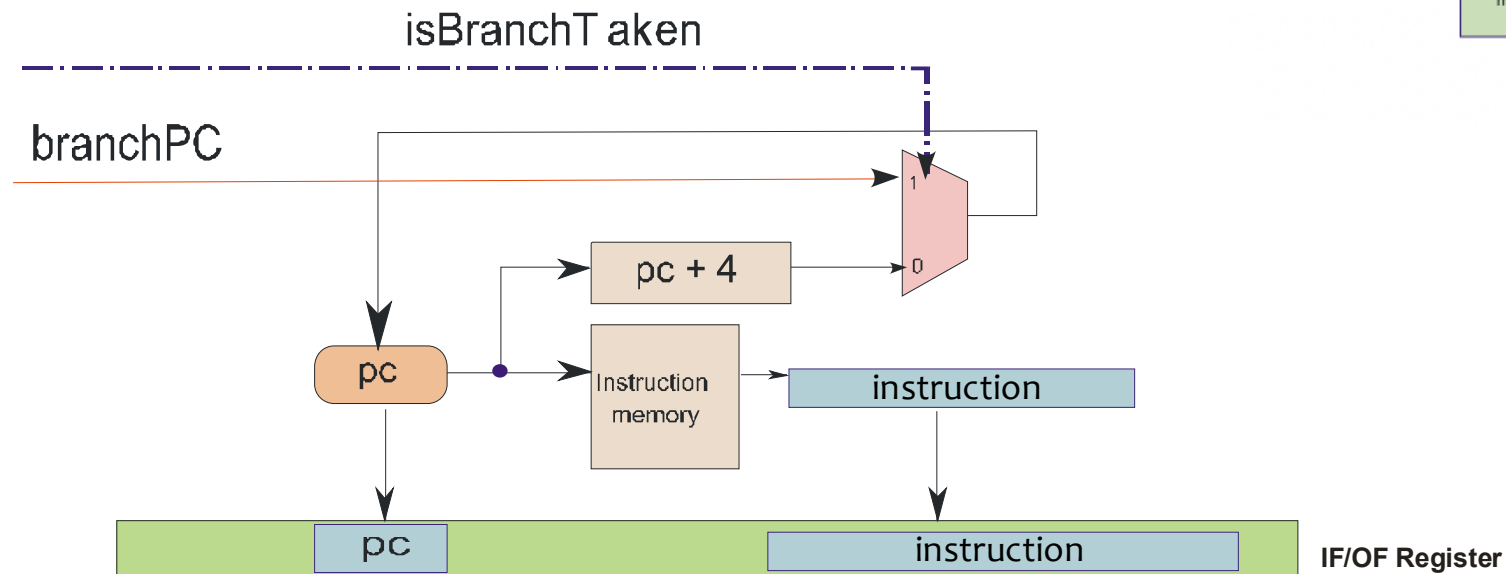
- * What travels **between stages** ?
 - * **ANSWER** : the instruction packet
- * Instruction Packet
 - * **Instruction contents**
 - * Program counter
 - * **All intermediate results**
 - * Control signals
- * Every instruction moves with its entire state, no interference between instructions

Outline

- * Overview of Pipelining
- * A Pipelined Data Path
- * Pipeline Hazards
- * Pipeline with Interlocks
- * Forwarding
- * Performance Metrics
- * Interrupts/ Exceptions

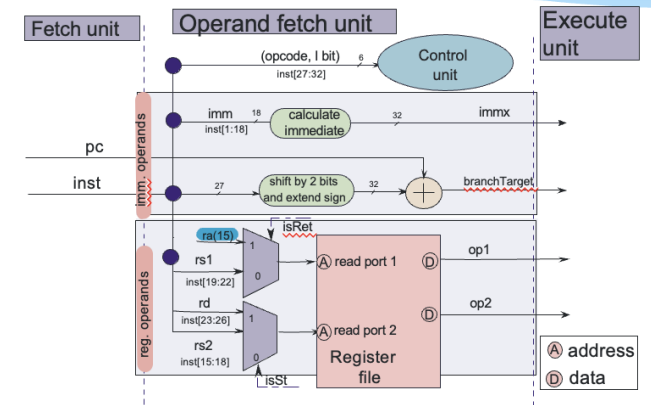
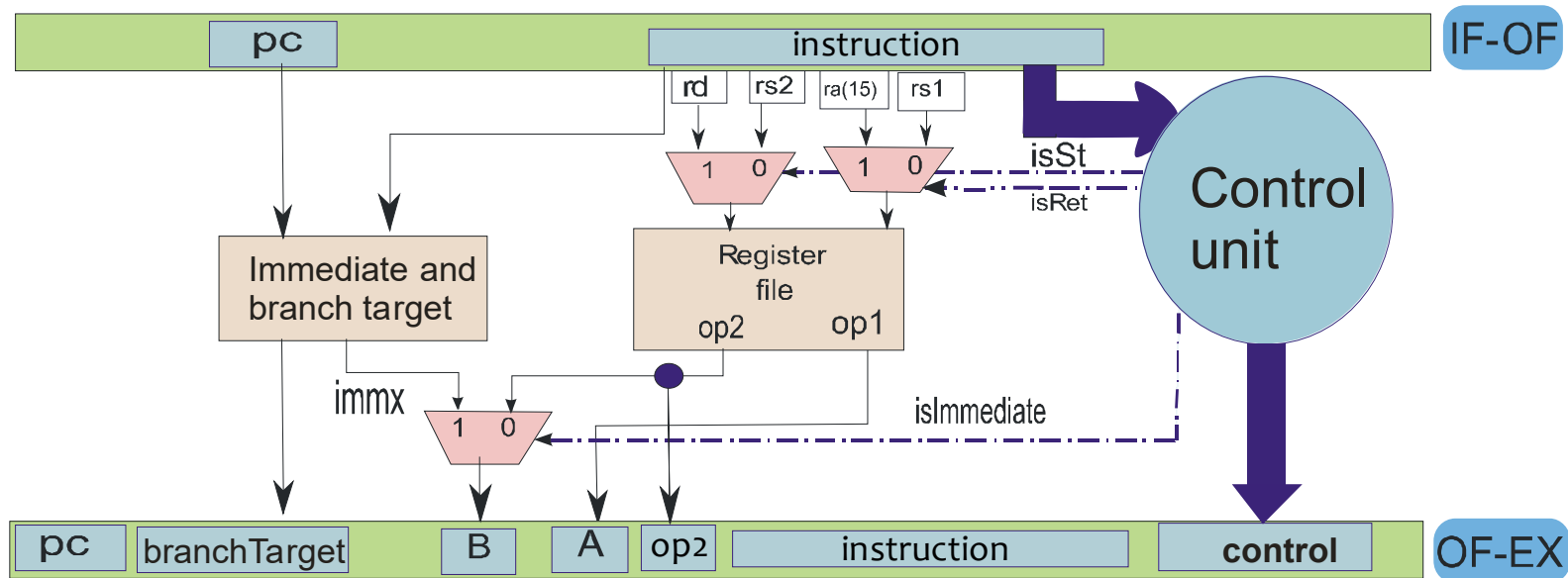


IF Stage



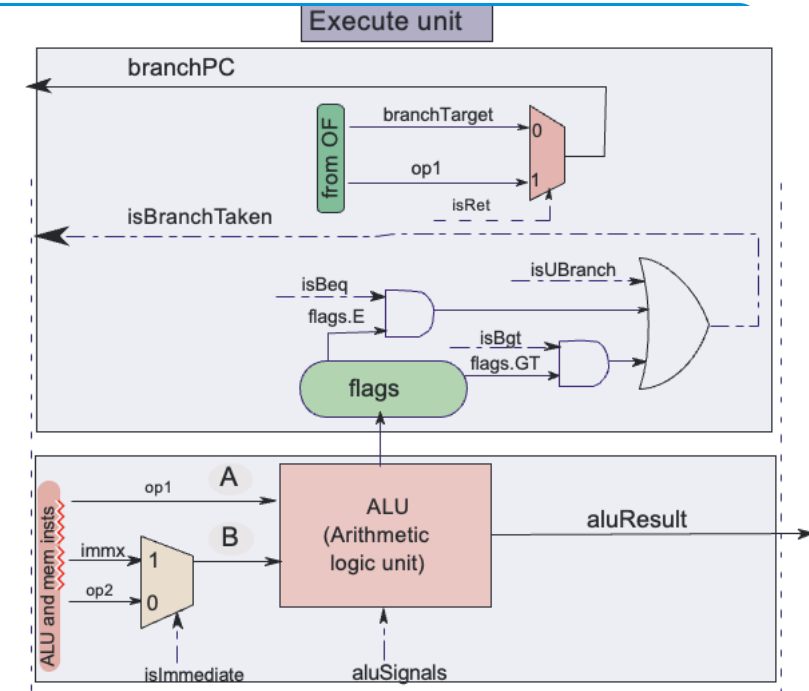
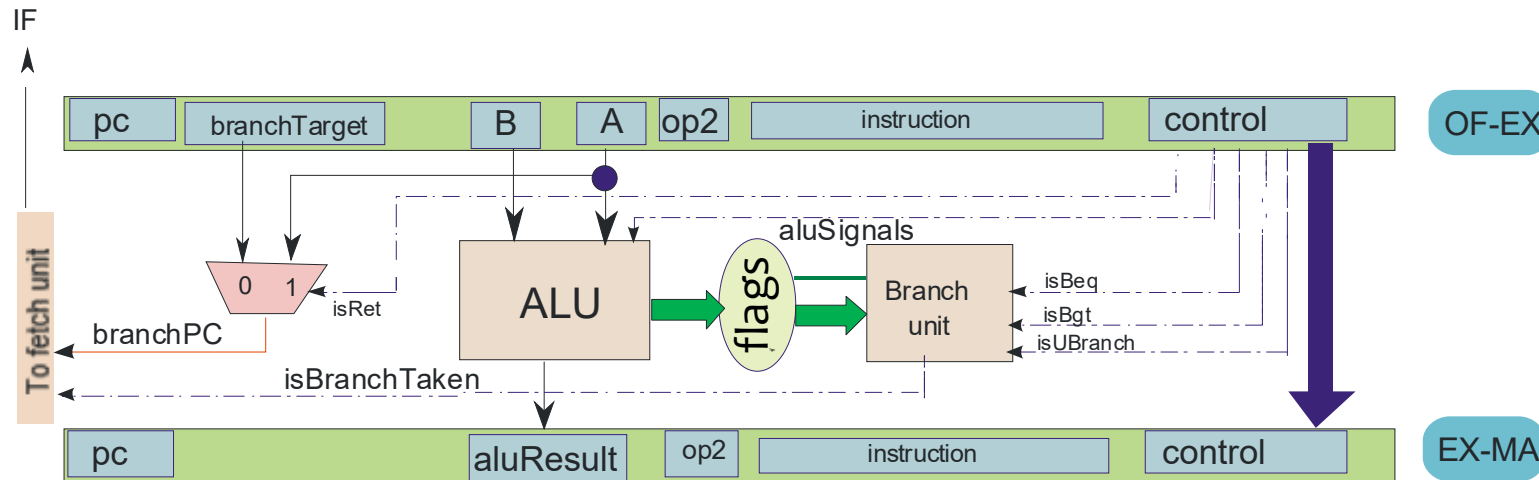
- * Instruction contents saved in the **instruction** field

OF Stage



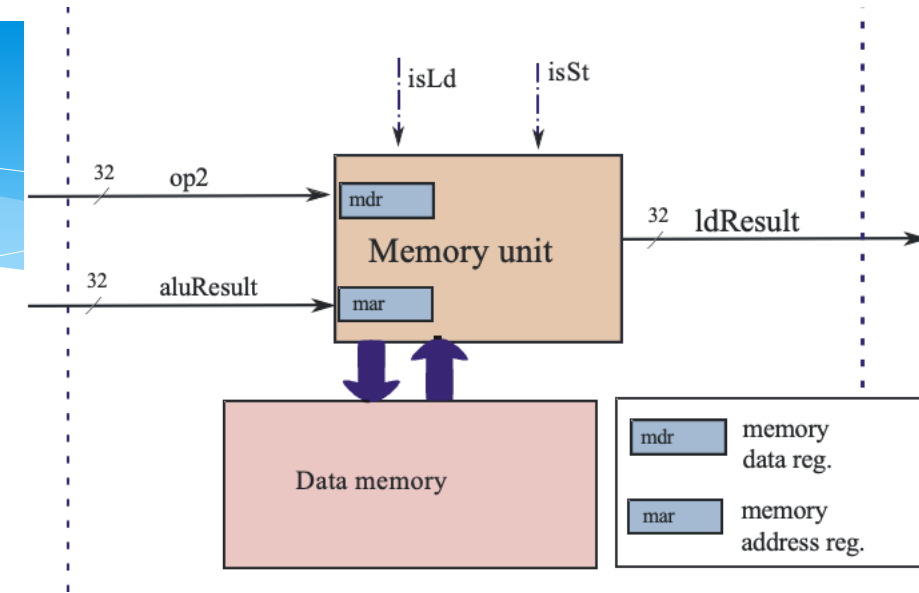
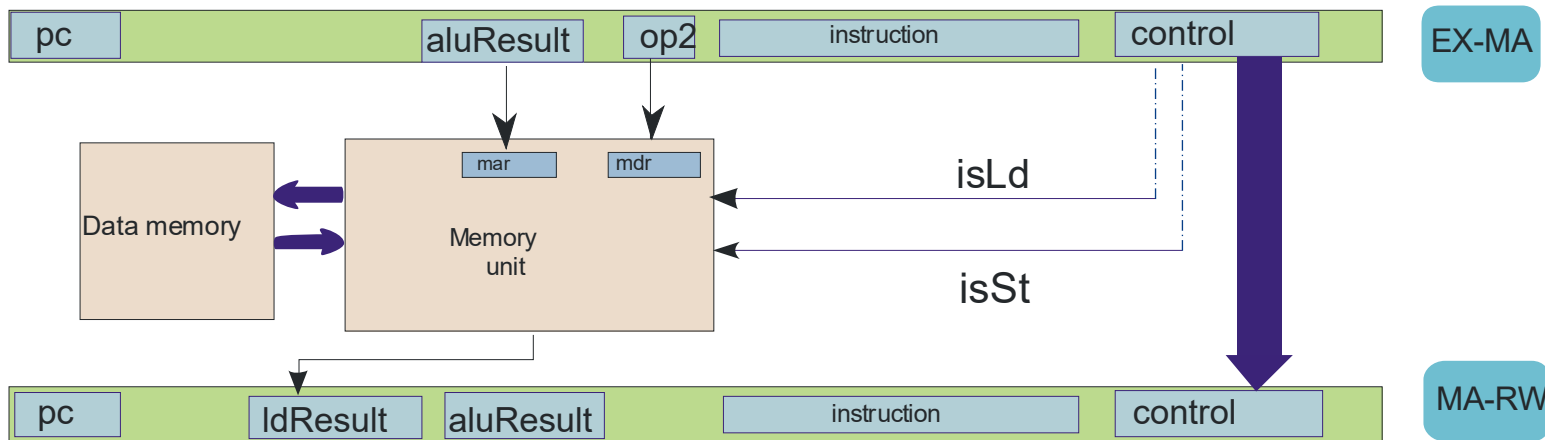
* **A, B** → ALU Operands, op2 (**store operand**),
control (**set of all control signals**)

EX Stage



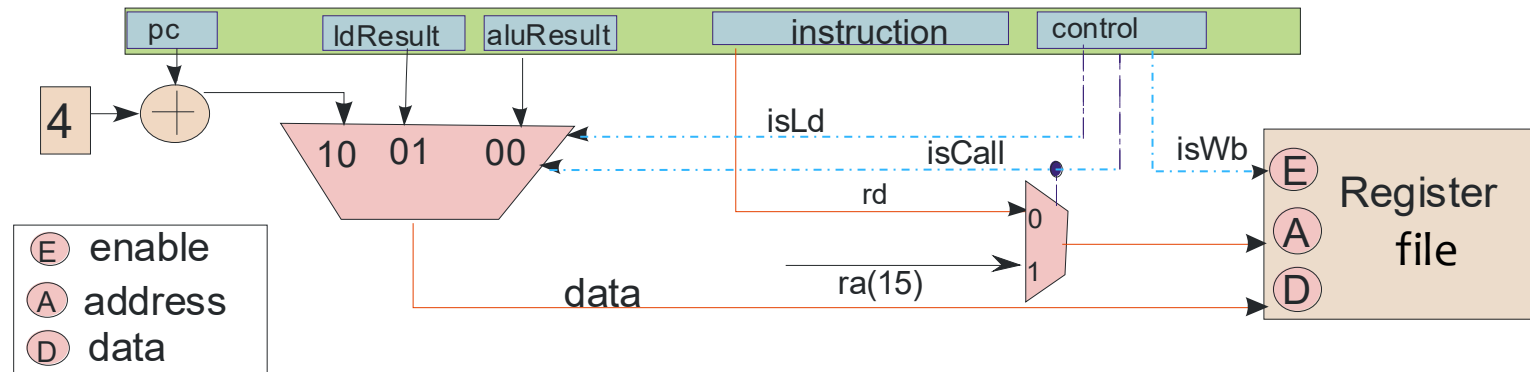
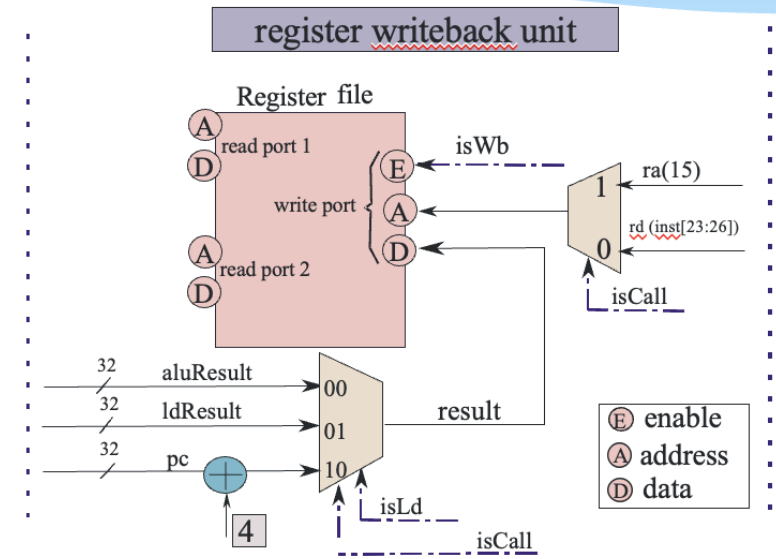
- * `aluResult` → result of the ALU Operation
- * `op2, control, pc, instruction` (passed from OF-EX)

MA Stage

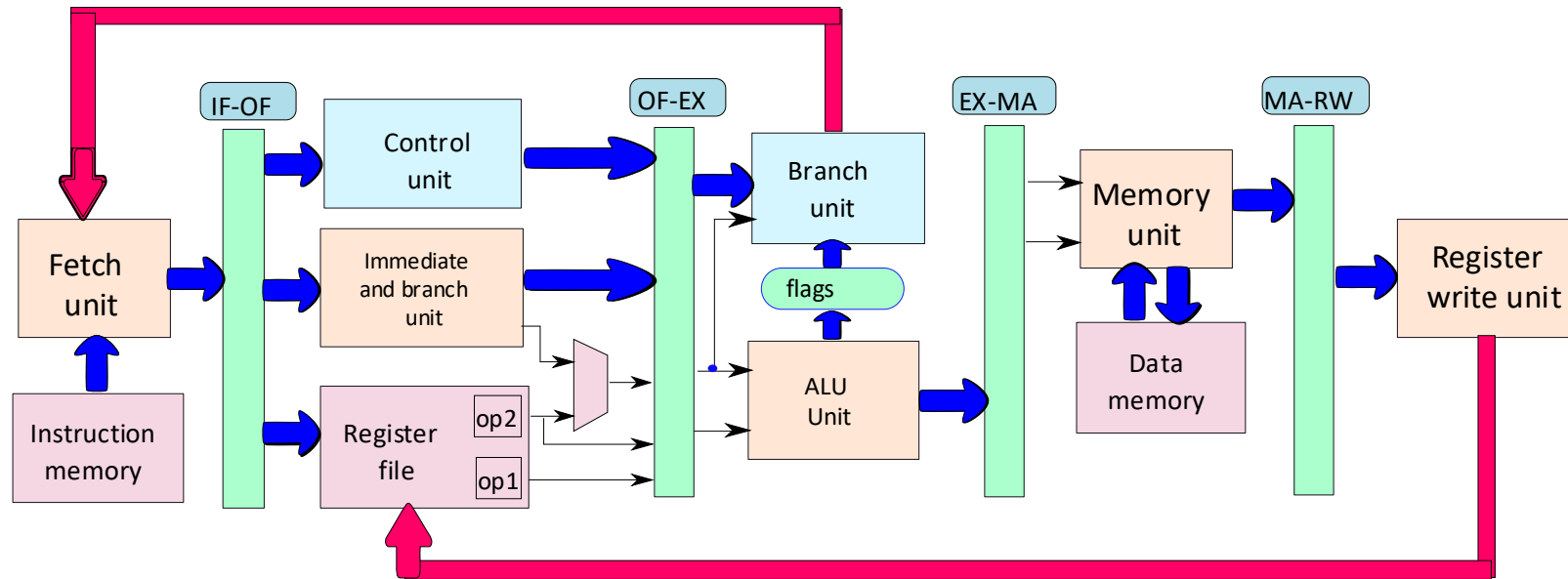


- * **IdResult** → result of the load operation
- * **aluResult, control, pc, instruction** (passed from EX-MA)

RW Stage



Abridged Diagram



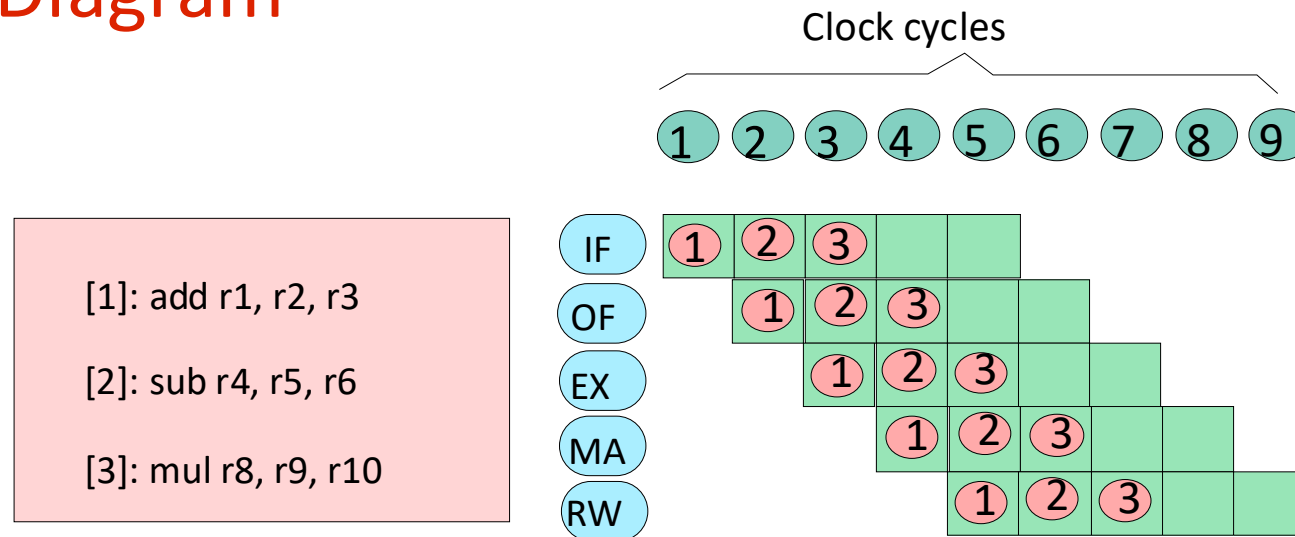
Outline

- * Overview of Pipelining
- * A Pipelined Data Path
- * Pipeline Hazards
- * Pipeline with Interlocks
- * Forwarding
- * Performance Metrics
- * Interrupts/ Exceptions



Pipeline Hazards

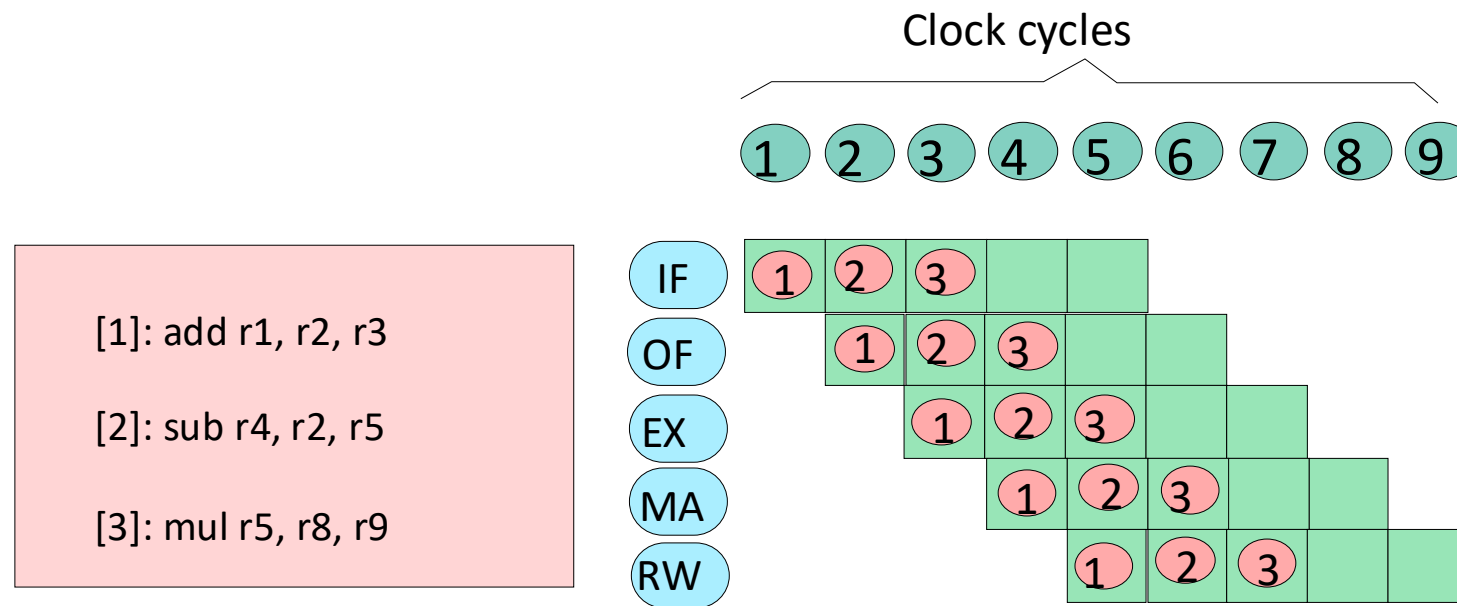
- * Now, let us consider **correctness**
- * Let us introduce a new tool → **Pipeline Diagram**



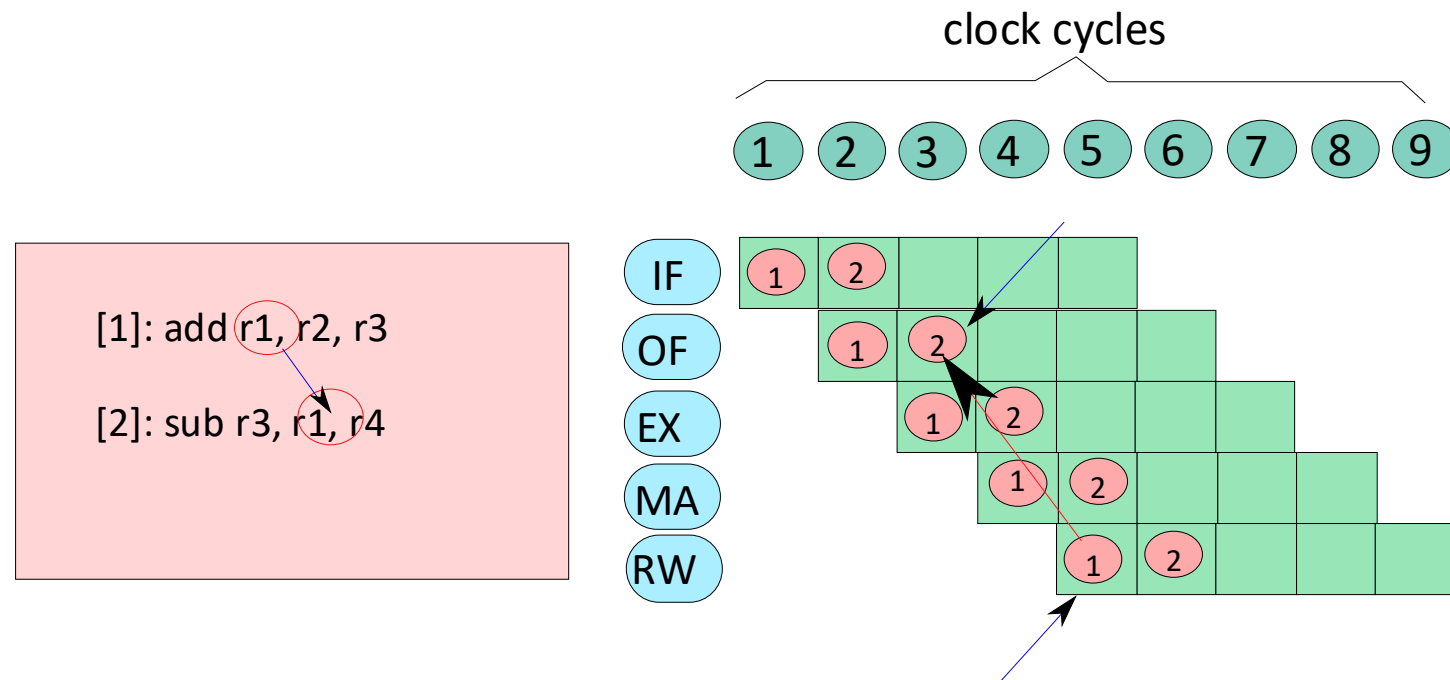
Rules for Constructing a Pipeline Diagram

- * It has 5 **rows**
 - * One per each stage
 - * The rows are **named** : IF, OF, EX, MA, and RW
- * Each **column** represents a clock cycle
- * Each **cell** represents the execution of an instruction in a stage
 - * It is **annotated** with the name(label) of the instruction
- * Instructions proceed from one stage to the next across clock cycles

Example



Data Hazards



* Instruction 2 will read incorrect values !!!

Data Hazard

Definition: A **hazard** is defined as the possibility of erroneous execution of an instruction in a pipeline. A data hazard represents the possibility of erroneous execution because of the unavailability of data, or the availability of **incorrect** data.

- * This situation represents a **data hazard**
- * In specific,
 - * it is a **RAW** (read after write) hazard
- * The earliest we can dispatch instruction 2, is cycle 5

Other Types of Data Hazards

- * Our pipeline is in-order

Definition: In an in-order pipeline (such as ours), a preceding instruction is always ahead of a succeeding instruction in the pipeline. Modern processors however use out-of-order pipelines that break this rule. It is possible for later instructions to execute before earlier instructions.

- * We will only have RAW hazards in our pipeline.
- * Out-of-order pipelines can have WAR and WAW hazards

WAW Hazards

```
[1]: add r1, r2, r3  
[2]: sub r1, r4, r3
```

- * Instruction [2] cannot write the value of r1, before instruction [1] writes to it, will lead to a WAW hazard

WAR Hazards

```
[1]: add r1, r2, r3  
[2]: add r2, r5, r6
```

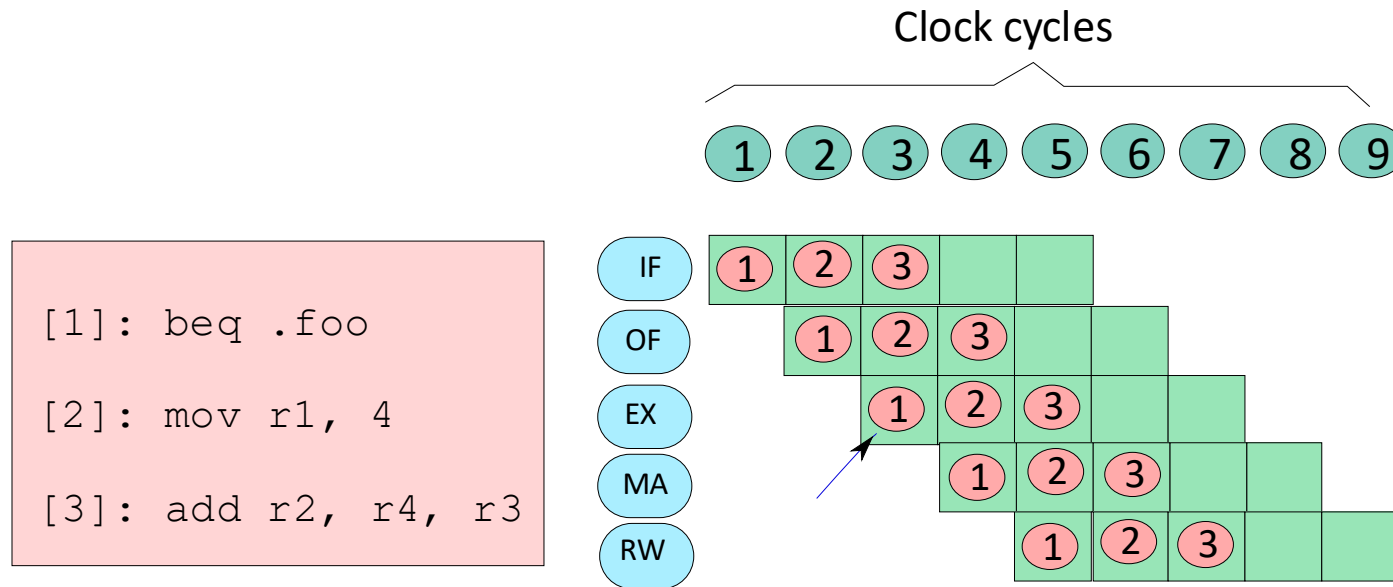
- * **Instruction** [2] cannot **write** the value of r2, before **instruction** [1] reads it → will lead to a **WAR** hazard

Control Hazards

```
[1]: beq .foo  
[2]: mov r1, 4  
[3]: add r2, r4, r3  
...  
...  
.foo:  
[100]: add r4, r1, r2
```

- * If the branch is **taken**, instructions [2] and [3], might get fetched, incorrectly

Control Hazard – Pipeline Diagram



- * The two **instructions** fetched immediately after a branch instruction might have been fetched **incorrectly**.

Control Hazards

- * The two **instructions** fetched immediately after a branch instruction might have been fetched **incorrectly**.
- * These instructions are said to be on the **wrong path**
- * A **control hazard** represents the **possibility** of **erroneous** execution in a **pipeline** because instructions in the **wrong path** of a **branch** can possibly get **executed** and save their results in memory, or in the register file

Structural Hazards

- * A **structural hazard** may occur when two instructions have a conflict on the same set of resources in a cycle
- * Example :
 - * Assume that we have an add instruction that can read one operand from memory
 - * `add r1, r2, 10[r3]`

Structural Hazards - II

```
[1]: st r4, 20[r5]  
[2]: sub r8, r9, r10  
[3]: add r1, r2, 10[r3]
```

- * This code will have a structural hazard
 - * [3] tries to read 10[r3] (MA unit) in cycle 4
 - * [1] tries to write to 20[r5] (MA unit) in cycle 4
- * Does not happen in our pipeline

Solutions in Software

* Data hazards

- * Insert **nop** instructions, reorder **code**

```
[1]: add r1, r2, r3 ✓ RAW  
[2]: sub r3, r1, r4
```

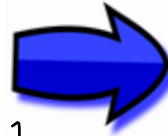


```
✓[1]: add r1, r2, r3 Finished ← Producer  
✓[2]: nop RW  
[3]: nop MA  
[4]: nop Ex  
[5]: sub r3, r1, r4 OF ← Consumer
```

Handwritten annotations: A double vertical line is drawn between instructions [2] and [4]. A curved arrow points from the 'OF' (Order Forward) label of instruction [5] back to the 'Finished' label of instruction [1].

Code Reordering

① add r1, r2, r3 RAW
② add r4, r1, 3
③ add r8, r5, r6 RAW
④ add r9, r8, r5
⑤ add r10, r11, r12 RAW
⑥ add r13, r10, 2



① add r1, r2, r3
③ add r8, r5, r6
⑤ add r10, r11, r12
nop
② add r4, r1, 3
④ add r9, r8, r5
⑥ add r13, r10, 2

Control Hazards

- * **Trivial Solution** : Add two nop instructions after every branch
- * **Better solution** :
 - * Assume that the two instructions fetched after a branch are **valid** instructions
 - * These instructions are said to be in the **delay slots**
 - * Such a branch is known as a **delayed branch**

Example with 2 Delay Slots

```
add r1, r2, r3
add r4, r5, r6
b .foo
add r8, r9, r10
```



```
b .foo
add r1, r2, r3
add r4, r5, r6
add r8, r9, r10
```

- * The **compiler** transfers **instructions** before the branch to the **delay slots**.
- * If it cannot find 2 valid **instructions**, it inserts **nops**.

Outline

- * Overview of Pipelining
- * A Pipelined Data Path
- * Pipeline Hazards
- * Pipeline with Interlocks
- * Forwarding
- * Performance Metrics
- * Interrupts/ Exceptions



Why interlocks ?

- * We cannot always **trust** the **compiler** to do a good job, or even introduce **nop** instructions correctly.
- * **Compilers** now need to be tailored to specific hardware.
- * We should ideally not expose the details of the **pipeline** to the **compiler** (might be confidential also)
- * Hardware mechanism to enforce correctness → **interlock**

Two kinds of Interlocks

- * Data-Lock

- * Do not allow a consumer instruction to move beyond the OF stage till it has read the correct values. Implication : Stall the IF and OF stages.

- * Branch-Lock

- * We never execute instructions in the wrong path.
- * The hardware needs to ensure both these conditions.

Conceptual Look at Pipeline with Interlocks

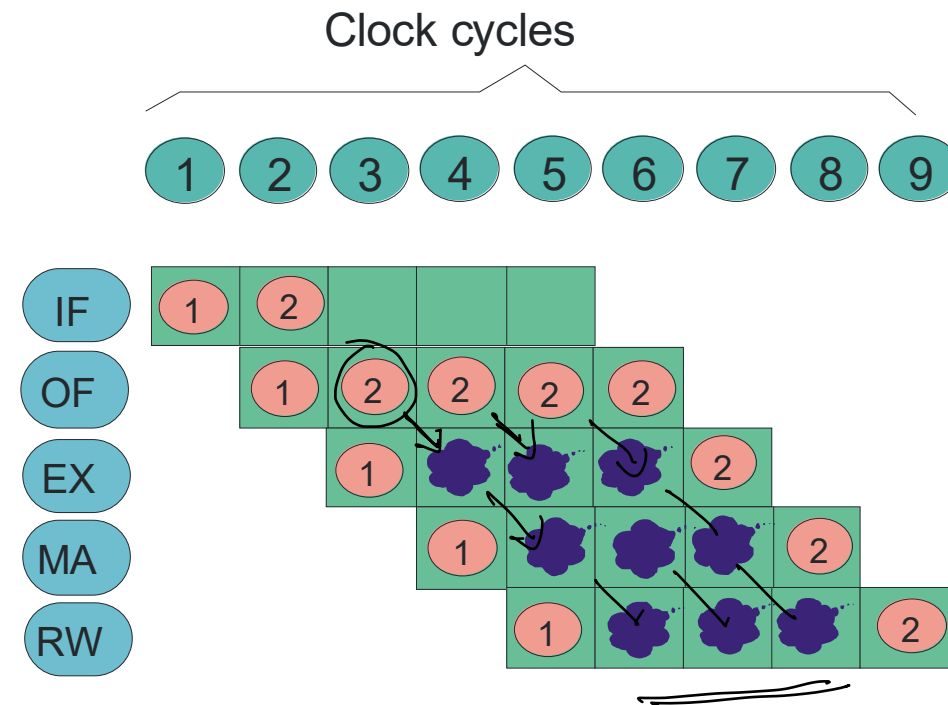
```
[1]: add r1, r2, r3  
[2]: sub r4, r1, r2
```

- * We have a **RAW hazard**
- * We need to **stall**, instruction [2] at the OF stage for 3 cycles.
- * We need to keep sending **nop** instructions to the **EX stage** during these 3 cycles

Example

 bubble

[1]: add r1, r2, r3
[2]: sub r4, r1, r2



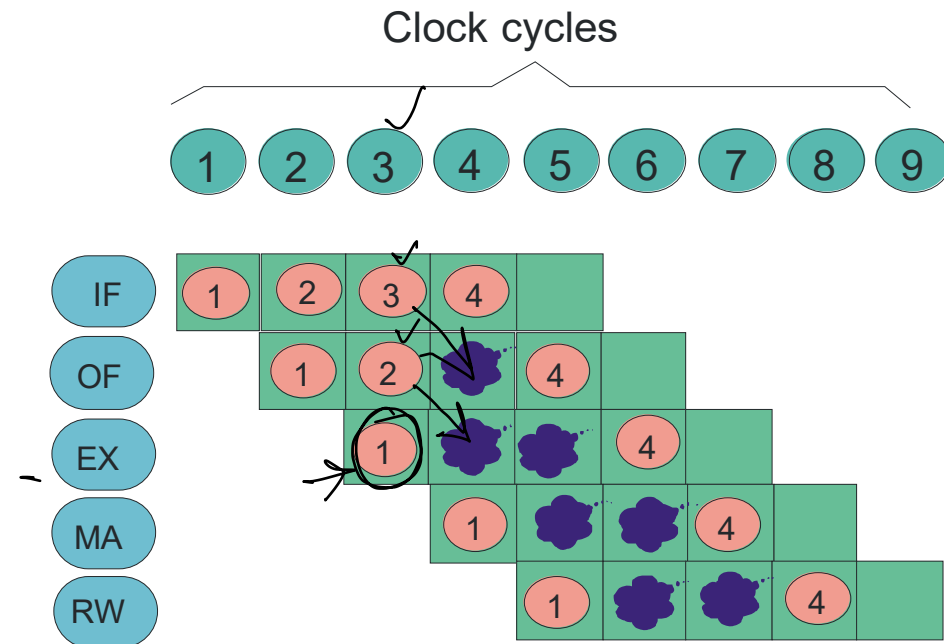
A Pipeline Bubble

- * A **pipeline bubble** is inserted into a stage, when the **previous stage** needs to be stalled
- * It is a **nop** instruction
- * To insert a **bubble**
 - * Create a **nop instruction packet**
 - * OR, Mark a designated **bubble bit** to 1

Bubbles in the Case of a Branch Instruction

 bubble

```
[1]: beq. foo ✓  
[2]: add r1, r2, r3 ↕  
[3]: sub r4, r5, r6  
....  
....  
.foo:  
[4]: add r8, r9, r10
```



Control Hazards and Bubbles

- * We know that an instruction is a **branch** in the ~~OF~~ stage
- * When it reaches the EX stage and the branch is taken, let us **convert** the instructions in the IF, and OF stages to **bubbles**
- * Ensures the **branch-lock** condition

Ensuring the Data-Lock Condition

- * When an **instruction** reaches the **OF** stage, check if it has a **conflict** with any of the instructions in the **EX, MA, and RW** stages
- * If there is **no conflict**, **nothing** needs to be done
- * Otherwise, **stall** the **pipeline** (IF and OF stages only)

Algorithm 5: Algorithm to detect conflicts between instructions

Data: instructions, [A], and [B]

Result: conflict exists (**true**), no conflict (**false**)

if [A].opcode $\in$ (nop, b, beq, bgt, call) **then**

 /* Does not read from any register

*/

return false

end

if [B].opcode $\in$ (nop, cmp, st, b, beq, bgt, ret) **then**

 /* Does not write to any register

*/

return false

end

 /* Set the sources

*/

src1 $\leftarrow$ [A].rs1

src2 $\leftarrow$ [A].rs2

if [A].opcode = st **then**

 src2 $\leftarrow$ [A].rd

end

if [A].opcode = ret **then**

 src1 $\leftarrow$ ra

end

hasSrc1 $\leftarrow$ **true**

if ([A] $\in$ (not, mov)) hasSrc1 $\leftarrow$ **false**

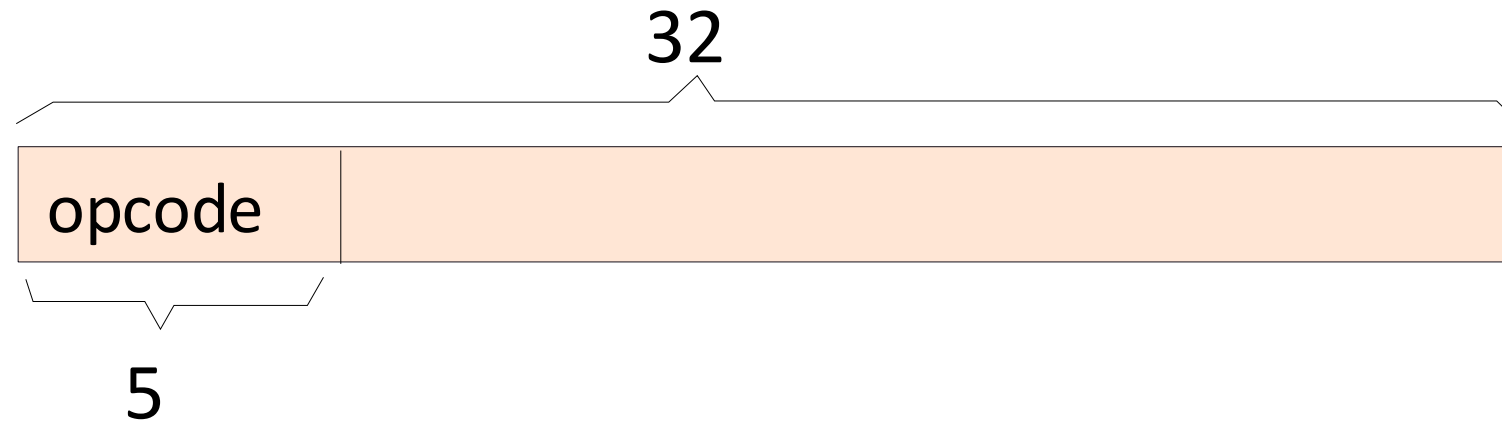
```
dest ← [B].rd
if [B].opcode = call then
    dest ← ra
end
/* Check the second operand to see if it is a register */
hasSrc2 ← true
if [A].opcode ≠ (st) then
    if [A].l = 1 then
        hasSrc2 ← false
    end
end
/* Detect conflicts */
if (hasSrc1 = true) and (src1 = dest) then
    return true
end
else if (hasSrc2 = true) and (src2 = dest) then
    return true
end
return false
```



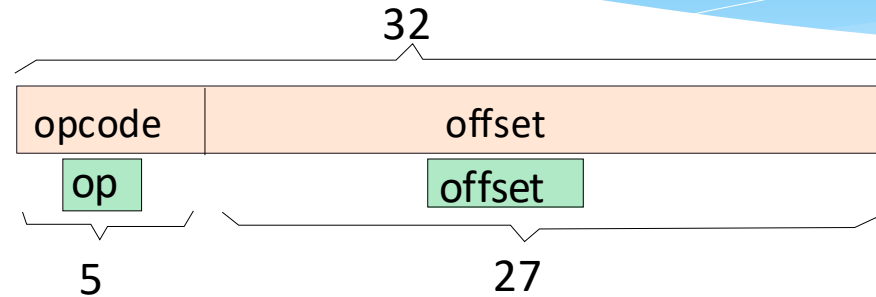
Recap

0-Address Instructions

* **nop** and **ret** instructions



1-Address Instructions

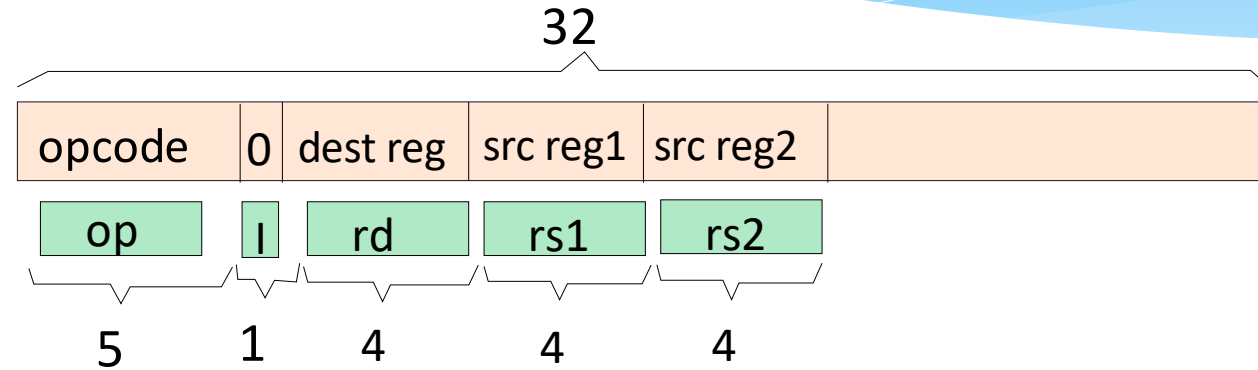


- * Instructions – **call, b, beq, bgt**
- * Use the **branch** format
- * Fields :
 - * 5 bit **opcode**
 - * 27 bit **offset** (PC relative addressing)
 - * Since the offset **points to a 4 byte word address**
 - * The **actual** address computed is : $PC + offset * 4$

3-Address Instructions

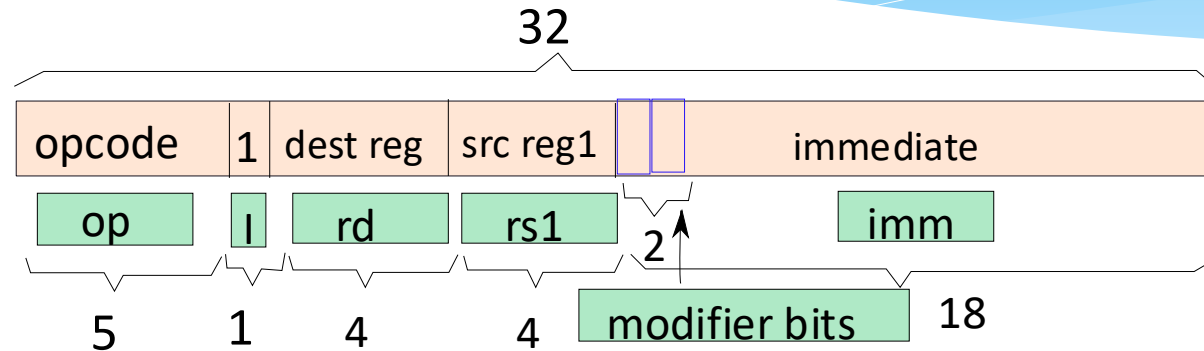
- * Instructions – add, sub, mul, div, mod, and, or, lsl, lsr, asr
- * Generic 3 address instruction
 - * <opcode> rd, rs1, <rs2/imm>
- * Let us use the I bit to specify if the second operand is an immediate or a register.
 - * I = 0 → second operand is a register
 - * I = 1 → second operand is an immediate
- * Since we have 16 registers, we need 4 bits to specify a register

Register Format



- * **opcode** → type of the instruction
- * **I** bit → 0 (second operand is a register)
- * **dest reg** → rd
- * **source register 1** → rs1
- * **source register 2** → rs2

Immediate Format



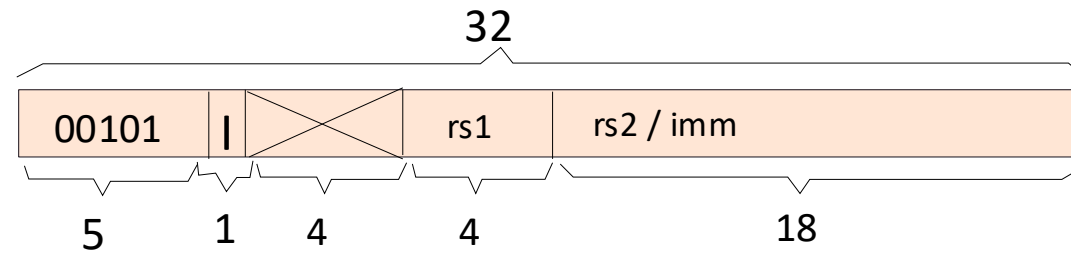
- * **opcode** → type of the instruction
- * **I** bit → 1 (second operand is an immediate)
- * **dest reg** → rd
- * **source register 1** → rs1
- * **Immediate** → imm
- * **modifier** bits → 00 (**default**), 01 (**u**), 10 (**h**)

2 Address Instructions

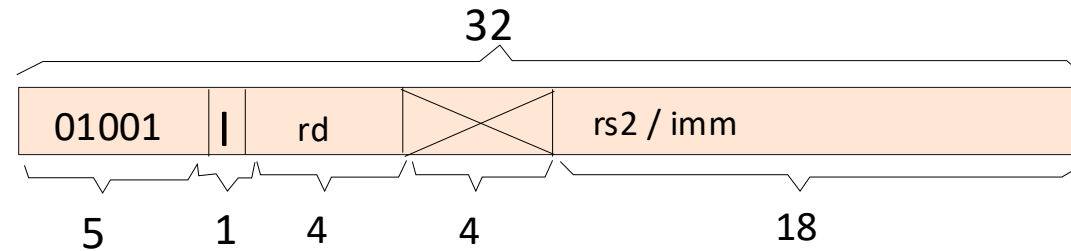
- * `cmp`, `not`, and `mov`
- * Use the 3 address : immediate or register formats
- * Do not use one of the fields

cmp, not, and mov

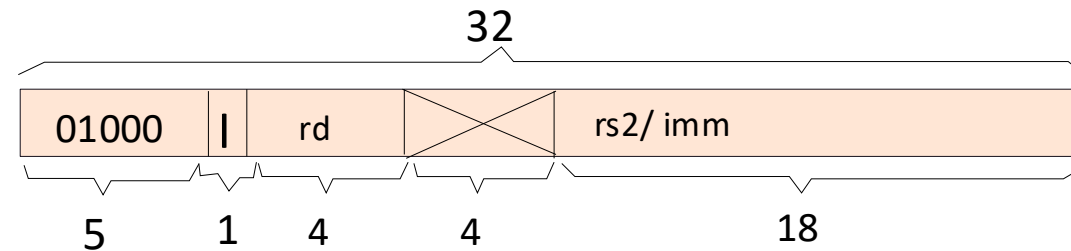
cmp



mov

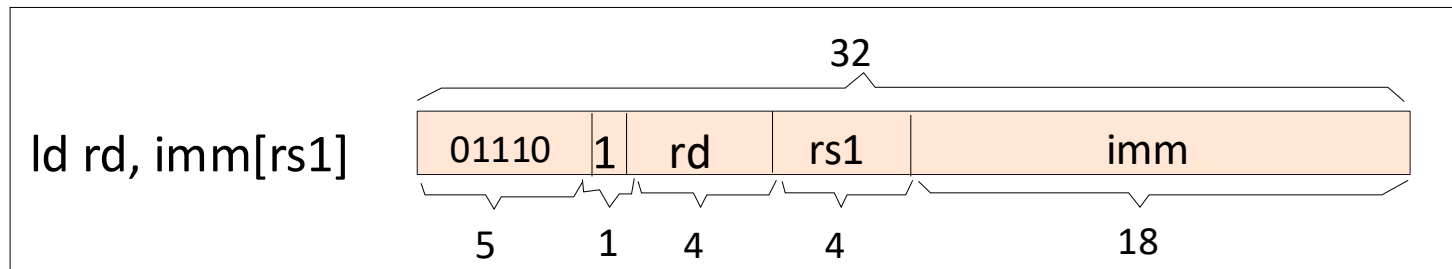


not



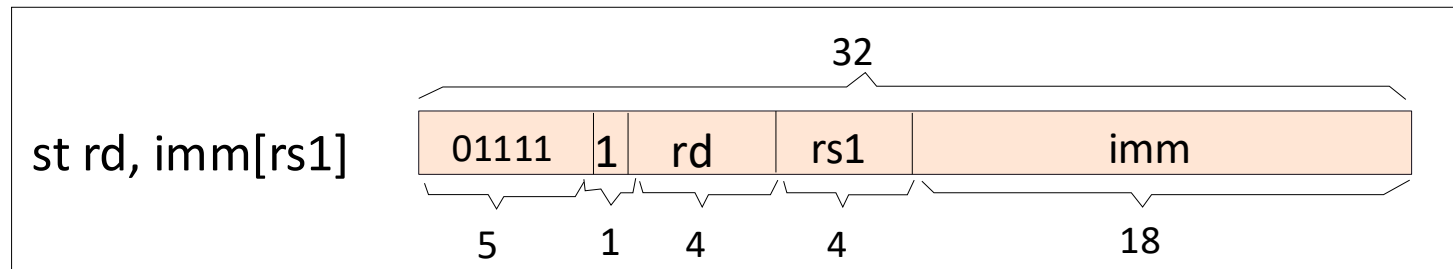
Load and Store Instructions

- * `ld rd, imm[rs1]`
- * `rs1` → base register
- * Use the **immediate** format.



Store Instruction

- * Let us **make an exception** and use the **immediate** format.
- * We use the **rd field** to save one of the **source registers**
- * **st rd, imm[rs1]**



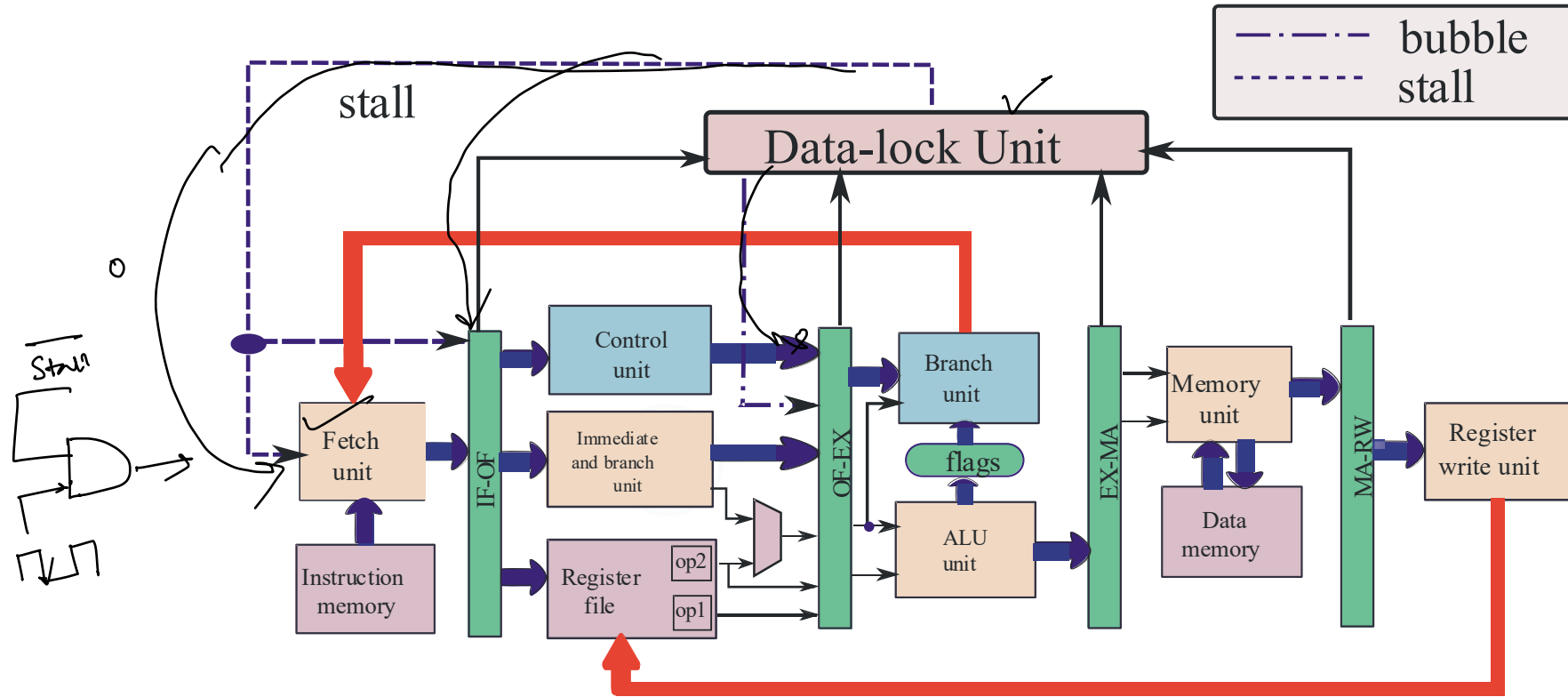


End of Recap

How to Stall a Pipeline ?

- * Disable the **write functionality** of :
 - * The **IF-OF** register
 - * and the **Program Counter (PC)**
- * To insert a **bubble**
 - * Write a **bubble** (**nop** instruction) into the **OF-EX** register

Data Path with Interlocks (Data-Lock)



Ensuring the Branch-Lock Condition

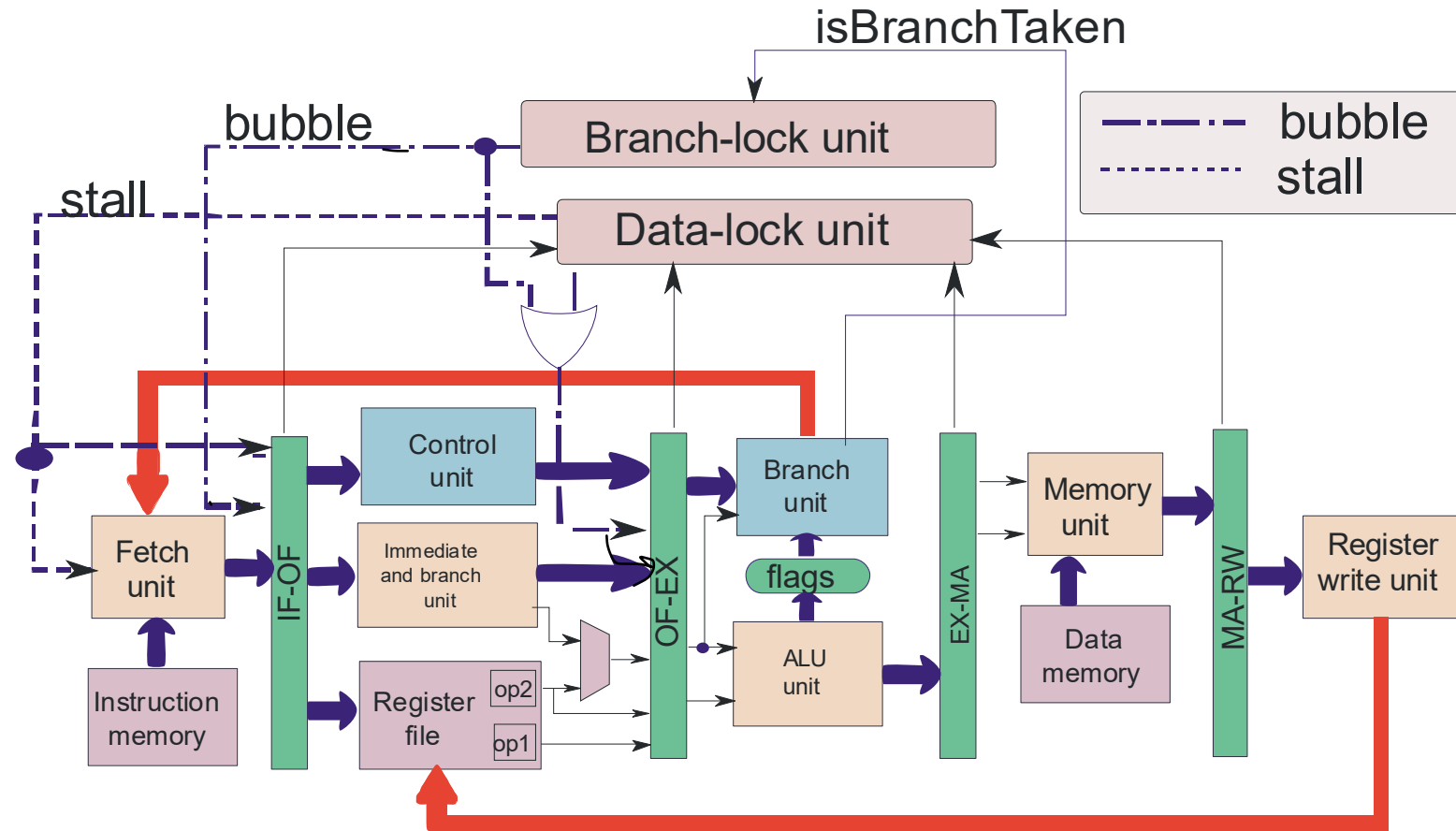
- * Option 1 :
 - * Use **delay slots** (**interlocks** not required)
- * Option 2 :
 - * **Convert** the instructions in the IF, and OF stages, to **bubbles** once a **branch instruction** reaches the **EX** stage.
 - * Start fetching from the **next PC (not taken)** or the **branch target (taken)**

Ensuring the Branch-Lock Condition - II

* Option 3

- * If the **branch instruction** in the **EX** stage is **taken**, then **invalidate** the instructions in the IF and OF stages. Start **fetching** from the **branch target**.
- * Otherwise, do not take **any special action**
- * **This method is also called predict not-taken (we shall use this method because it is more efficient than option 2)**

Data Path with Interlocks

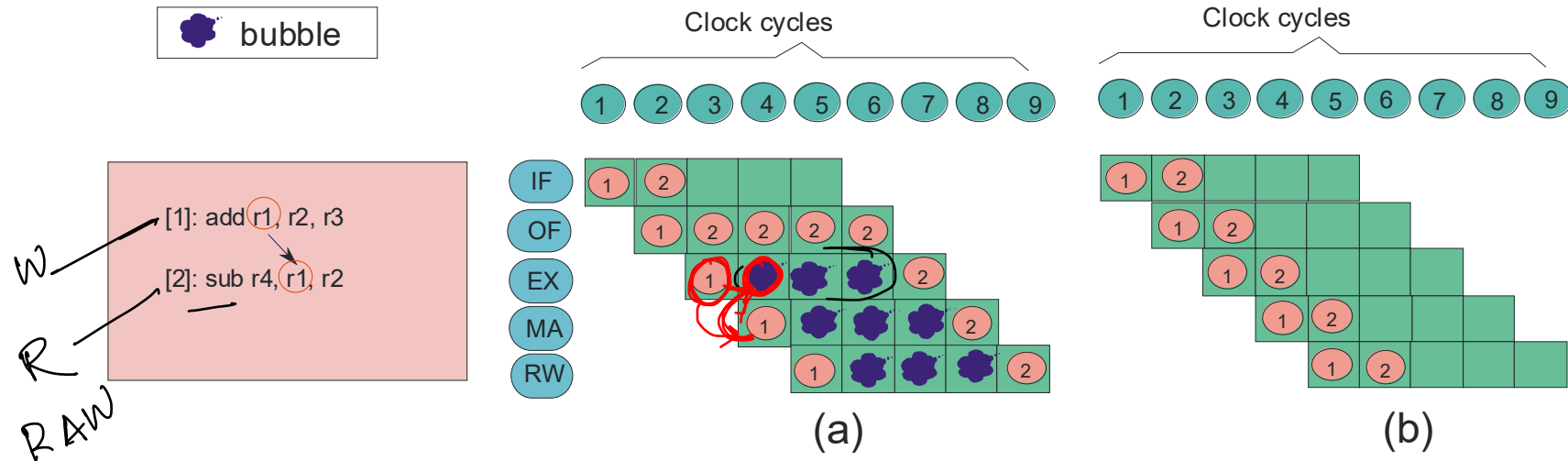


Outline

- * Overview of Pipelining
- * A Pipelined Data Path
- * Pipeline Hazards
- * Pipeline with Interlocks
- * Forwarding
- * Performance Metrics
- * Interrupts/ Exceptions



Relook at the Pipeline Diagram

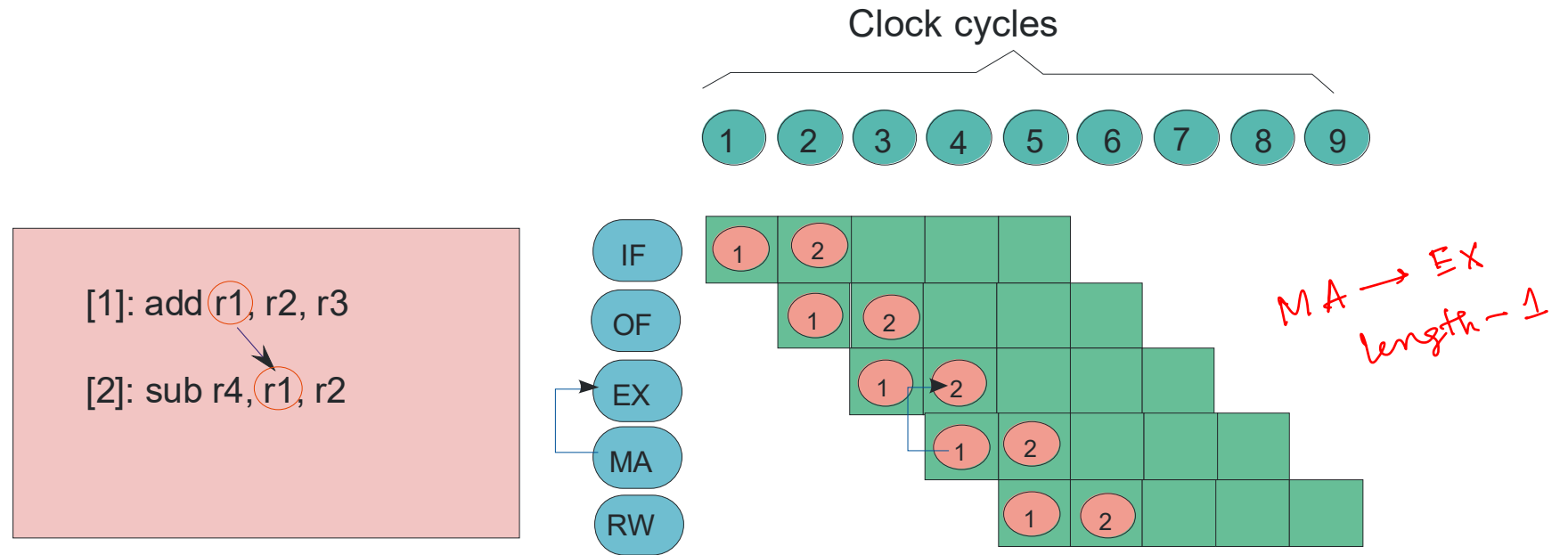


- * (a) → with bubbles
- * (b) → no bubbles (may lead to wrong results)

Crucial Insight (Figure (b))

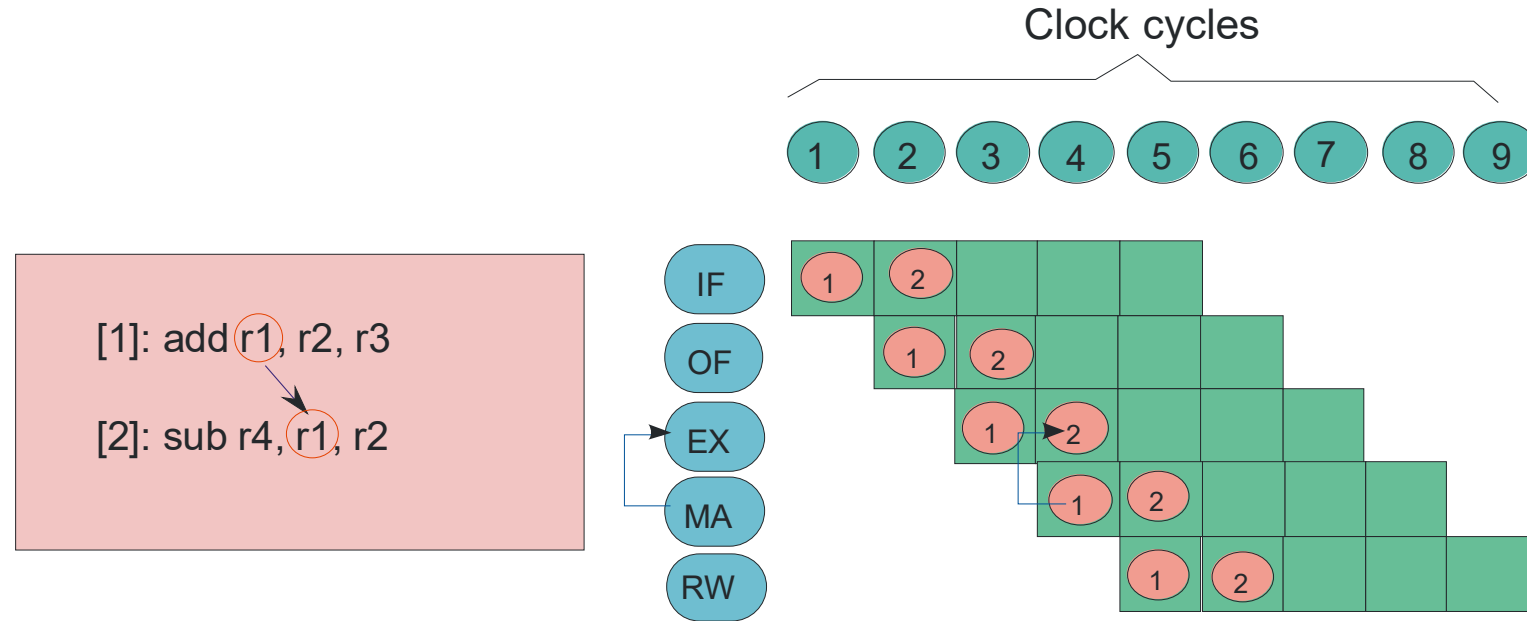
- * When does instruction 2 need the value of r1 ?
 - * **ANSWER** : Cycle 3, OF Stage (**wrong!!!**)
 - * **CORRECT ANSWER** : Cycle 4, EX Stage ✓
- * When does instruction 1 produce the value of r1 ?
 - * **ANSWER** : Cycle 5, RW Stage (**wrong!!!**)
 - * **CORRECT ANSWER** : End of Cycle 3, EX Stage

Forwarding



- * If the **correct value** is already there in another stage, we can **forward** it.

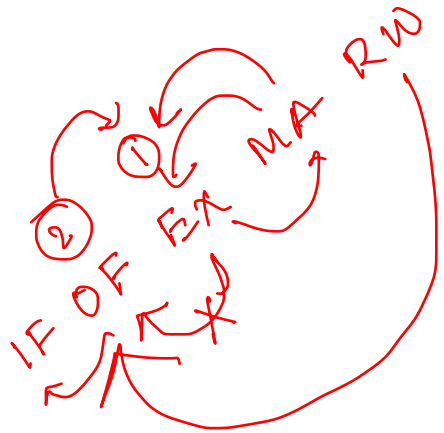
Forwarding from MA to EX



- * Forwarding in cycle 4 from instruction [1] to [2]

Different Forwarding Paths

①
②



- * We need to add a multitude of forwarding paths
- * Rules for creating forwarding paths
 - * Add a path from a later stage to an earlier stage
 - * Try to add a forwarding path as late as possible. For, example, we avoid the $EX \rightarrow OF$ forwarding path, since we have the $MA \rightarrow EX$ forwarding path
 - * The IF stage is not a part of any forwarding path.

Forwarding Path

- * 3 Stage Paths

- * RW $\rightarrow$ OF

- * 2 Stage Paths

- * RW $\rightarrow$ EX

- * MA $\rightarrow$ OF (**X Not Required**)

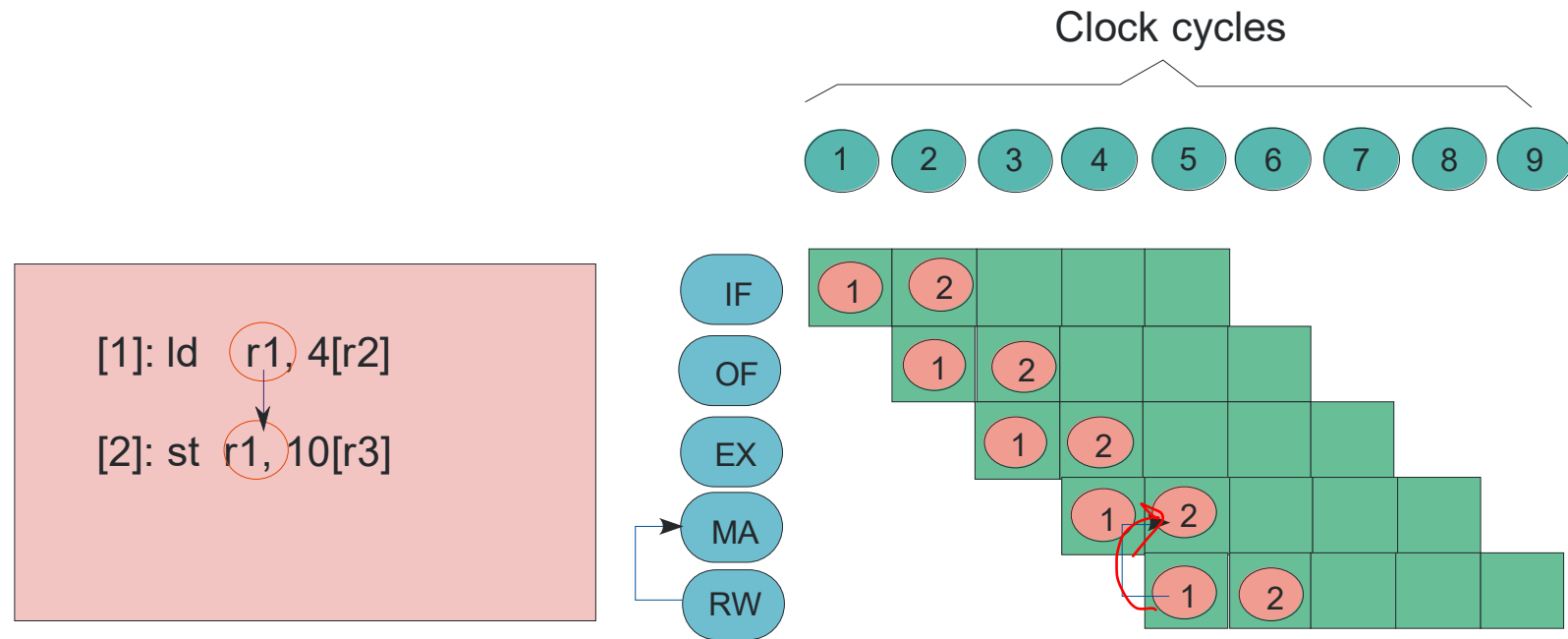
- * 1 Stage Paths

- * RW $\rightarrow$ MA (load to store)

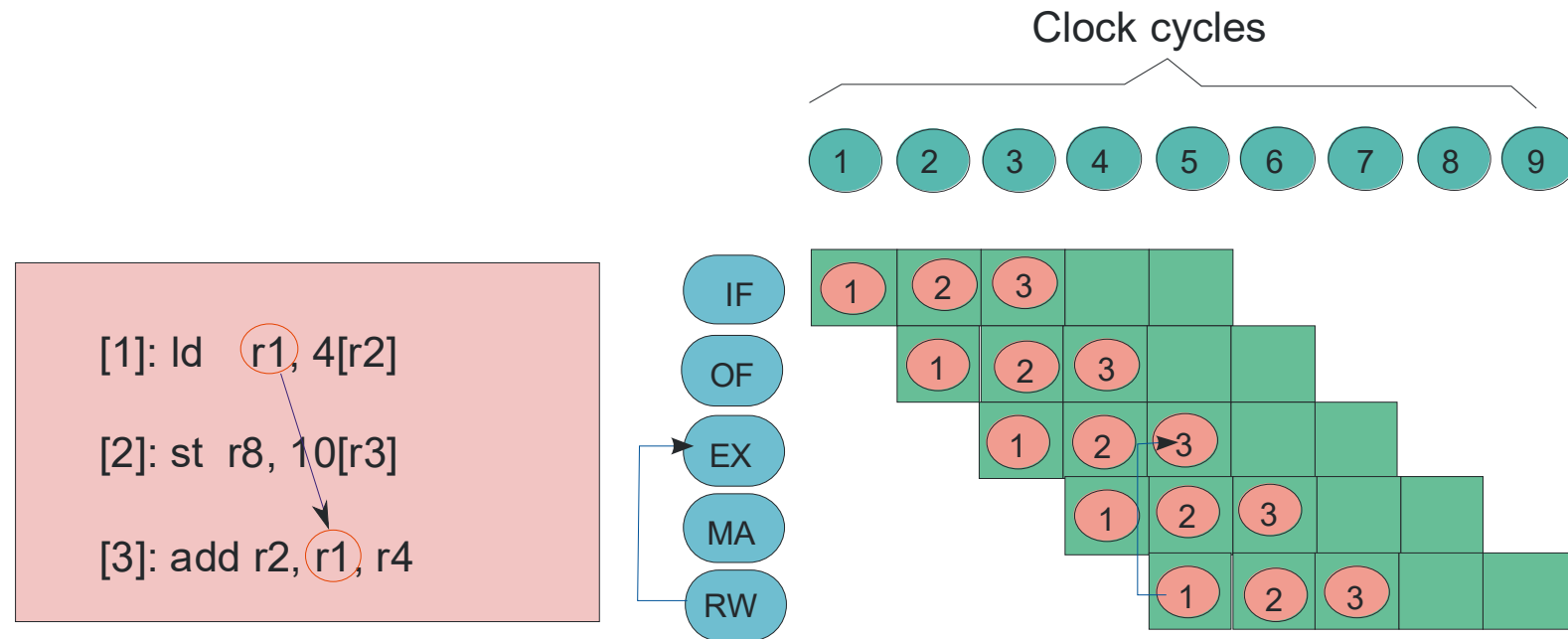
- * MA $\rightarrow$ EX (ALU Instructions, load, store)

- * EX $\rightarrow$ OF (**X Not Required**)

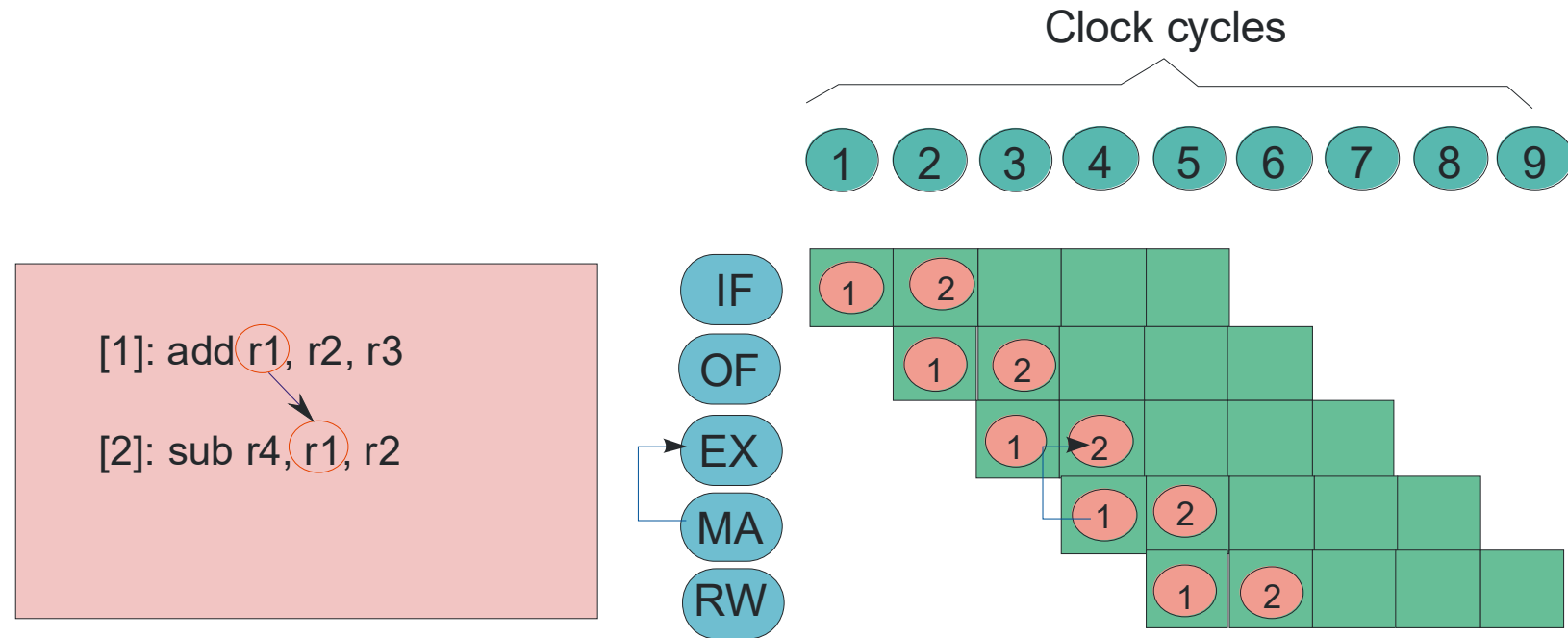
Forwarding Paths : RW → MA



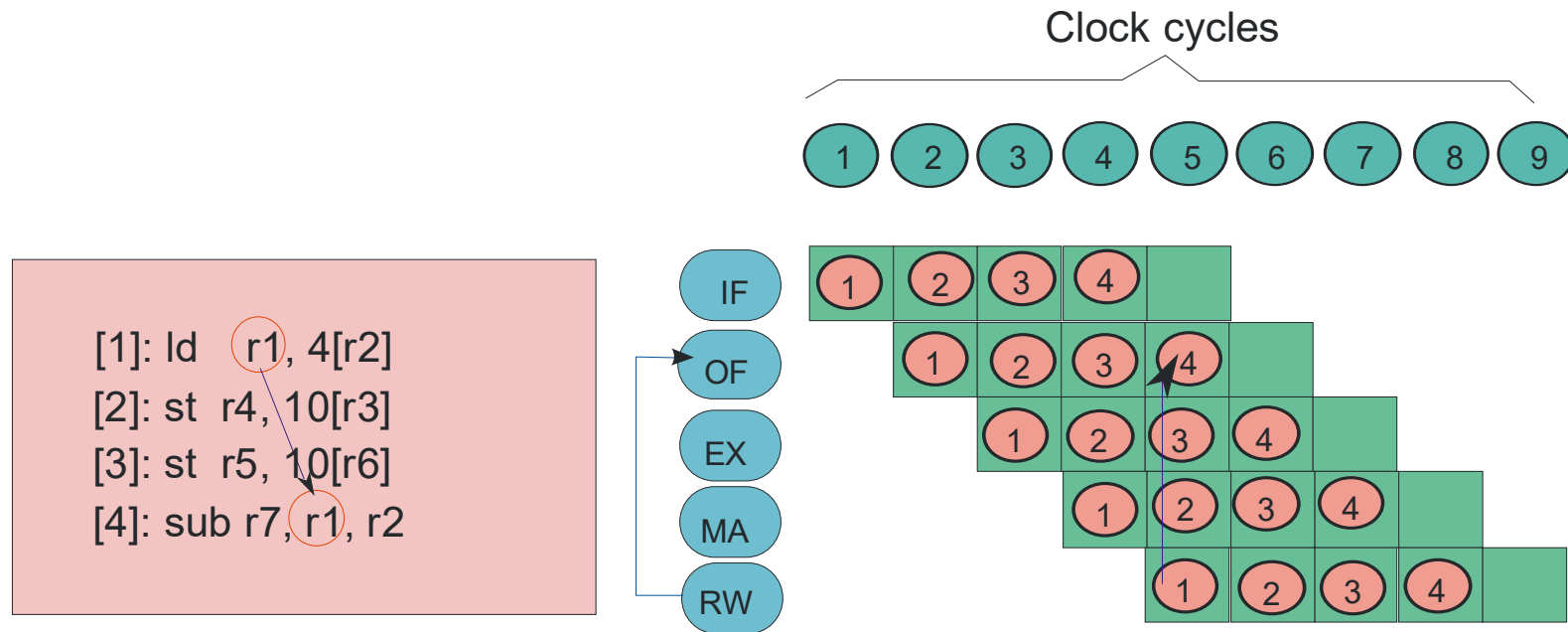
Forwarding Paths : RW $\rightarrow$ EX



Forwarding Path : MA $\rightarrow$ EX



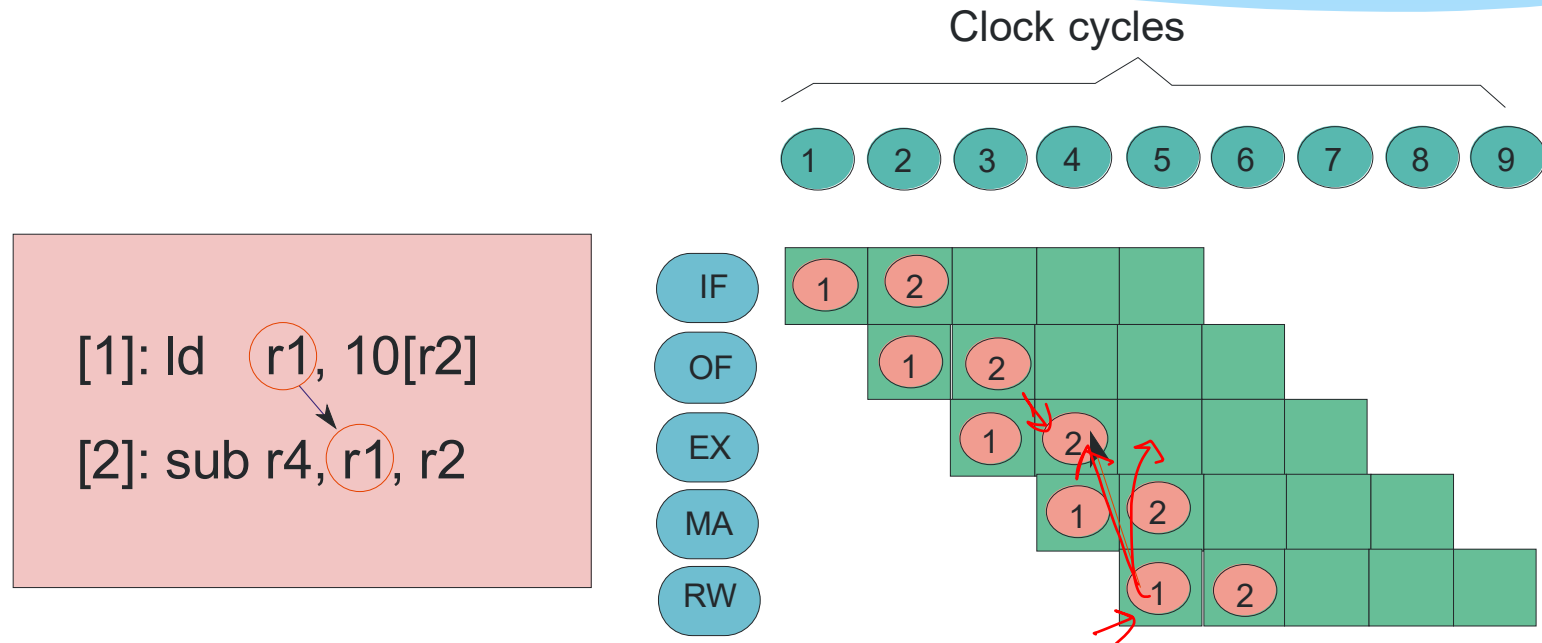
Forwarding Path : RW → OF



Data Hazards with Forwarding

- * Forwarding has unfortunately not eliminated all data hazards
- * We are left with one special case.
- * Load-use hazard
 - * The instruction immediately after a load instruction has a RAW dependence with it.

Load-Use Hazard

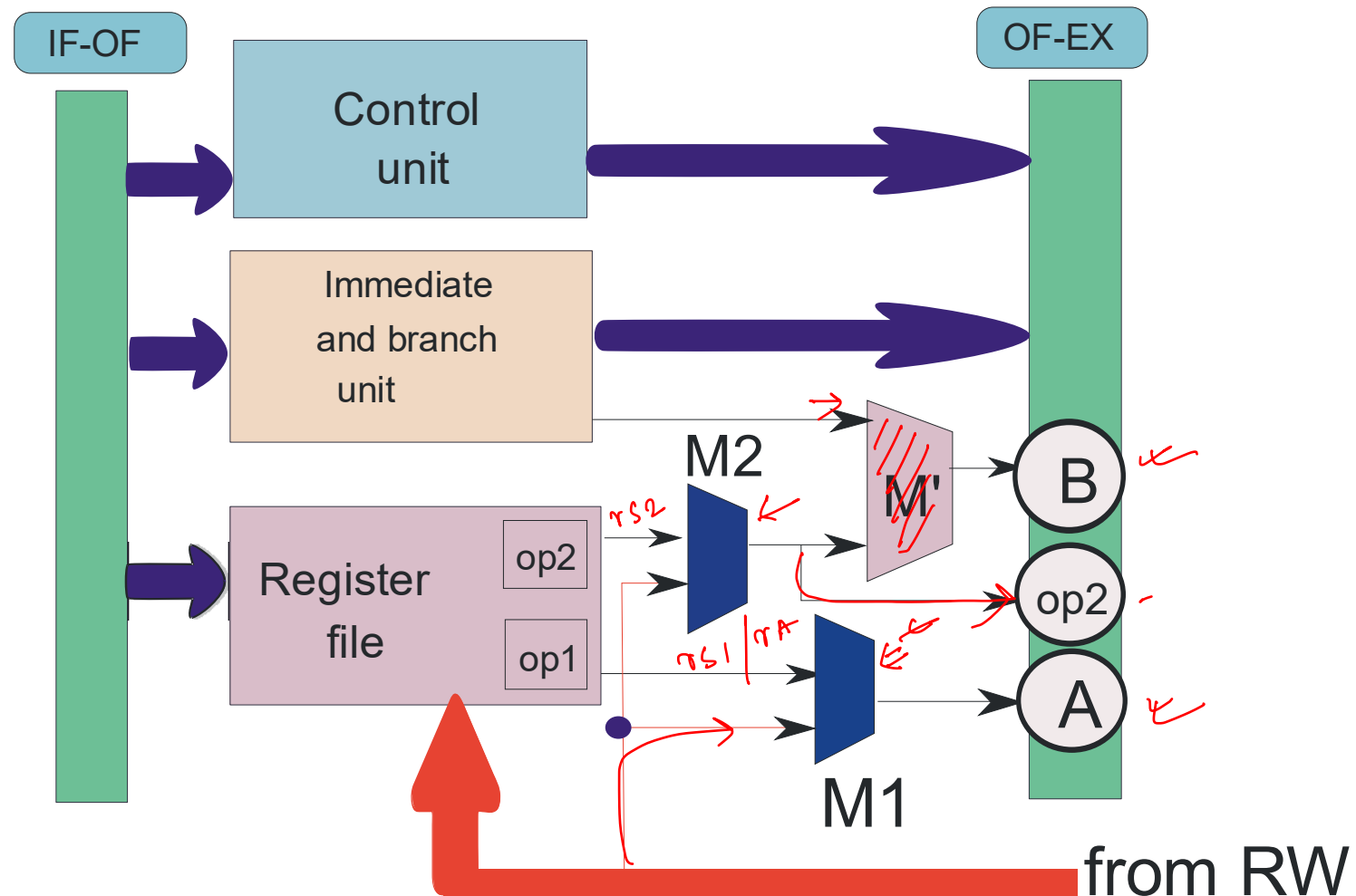


- * **Cannot forward** (arrow goes backwards in time)
- * Need to add a bubble (then use RW → EX forwarding)

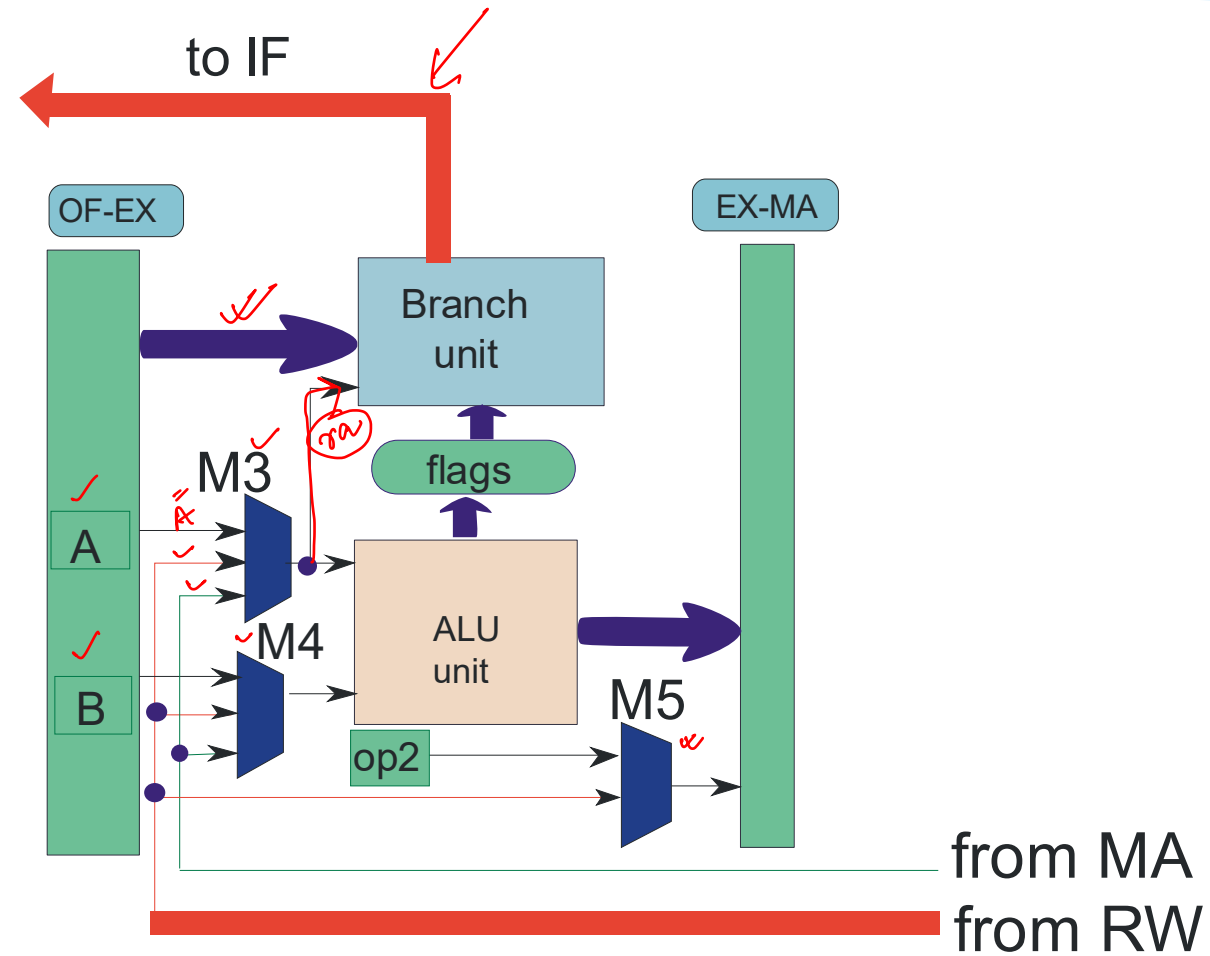
Implementation of Forwarding

- * At every stage there is a **choice** of inputs for each **functional unit**
 - * Use the **default** inputs from the previous stage
 - * **OR**, use one of the **forwarded** inputs
 - * Use a **multiplexer** to choose between the inputs
 - * A dedicated **forwarding unit** generates the signals for the **forwarding multiplexers**

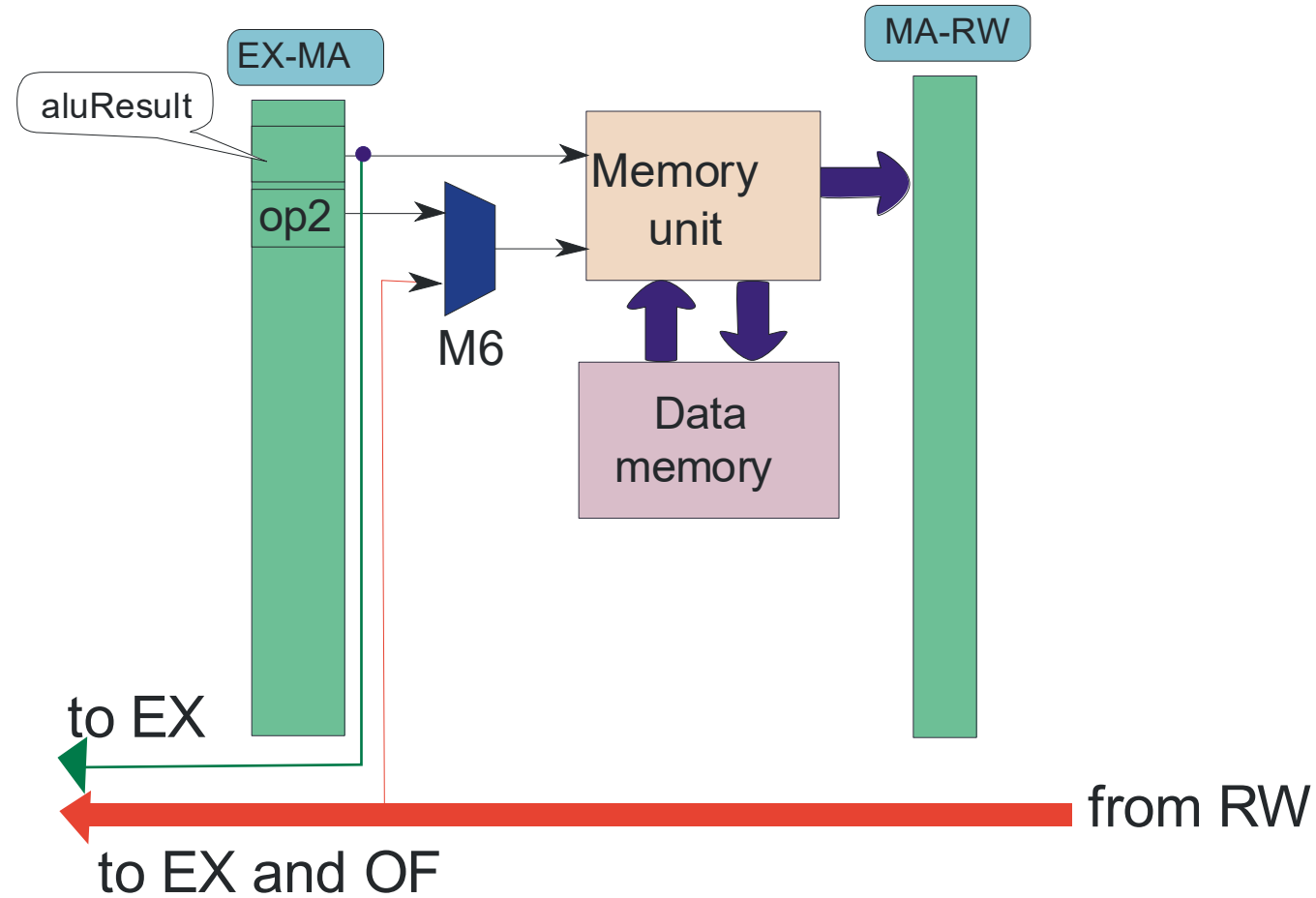
OF Stage with Forwarding



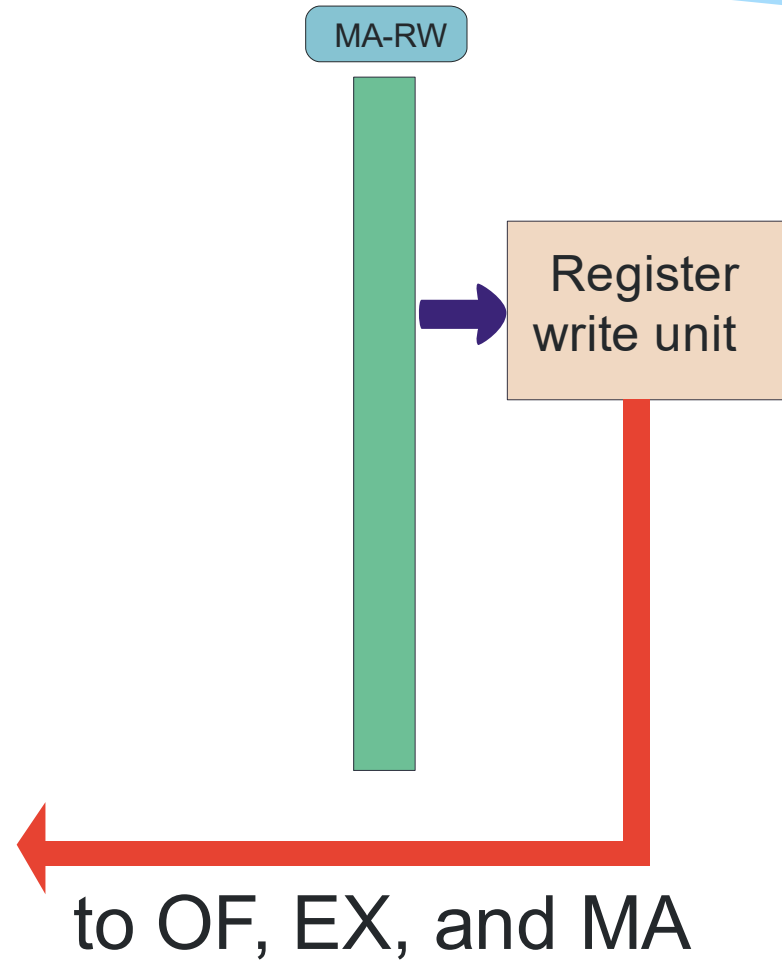
EX Stage with Forwarding



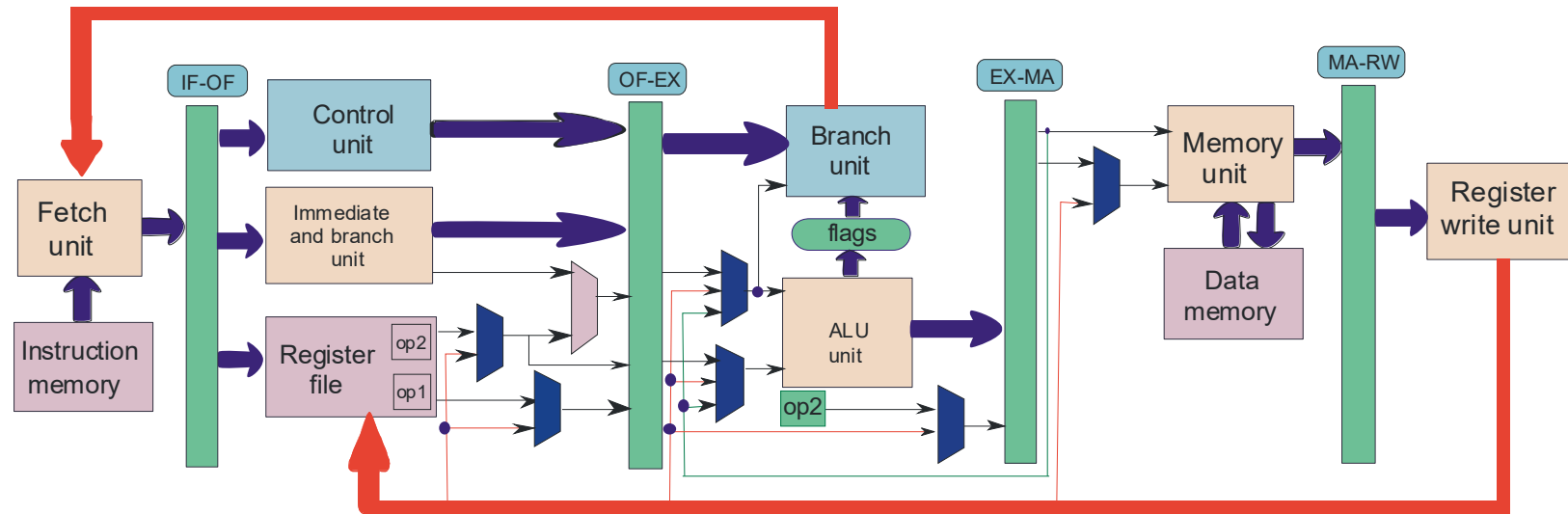
MA Stage with Forwarding



RW Stage with Forwarding



Data Path with Forwarding



Forwarding Conditions

- * Determine if there is a **conflict** between two **instructions** in different **stages**
 - * Find if there is a **conflict** for the first operand (rs1/ ra)
 - * Find if there is a **conflict** for the second operand (rs2/rd)
- * Always **forward** from the **latest** instruction (that is earlier than the current instruction)

Algorithm 6: Conflict on the first operand (rs1/ra)

Data: instructions, [A], and [B] (possible forwarding: [B] → [A])

Result: conflict exists on rs1/ra (**true**), no conflict (**false**)

if [A].opcode ∈ (nop, b, beq, bgt, call, not, mov) **then** 

/* Does not read from any register */

return false

end

if [B].opcode ∈ (nop, cmp, st, b, beq, bgt, ret) **then**

/* Does not write to any register */

return false

end

/* Set the sources */

src1 ← [A].rs1

if [A].opcode = ret **then**

src1 ← ra

end

/* Set the destination */

dest ← [B].rd

if [B].opcode = call **then**

dest ← ra

end

/* Detect conflicts */

if src1 = dest **then**

return true

end

return false

Algorithm 7: Conflict on the second operand (rs2/rd)

Data: instructions, [A], and [B] (possible forwarding: [B] $\rightarrow$ [A])

Result: conflict exists on second operand (rs2/rd) (**true**), no conflict (**false**)

```
if [A].opcode  $\in$  (nop, b, beq, bgt, call) then
    /* Does not read from any register */
    return false
end
if [B].opcode  $\in$  (nop, cmp, st, b, beq, bgt, ret) then
    /* Does not write to any register */
    return false
end
/* Check the second operand to see if it is a register */
if [A].opcode  $\neq$  (st) then
    if [A].I = 1 then
        return false
    end
end
/* Set the sources */
src2  $\leftarrow$  [A].rs2
if [A].opcode = st then
    src2  $\leftarrow$  [A].rd
end
```

```
/* Set the destination */  
dest  $\leftarrow$  [B].rd  
if [B].opcode = call then  
    dest  $\leftarrow$  ra  
end  
/* Detect conflicts */  
if src2 = dest then  
    return true  
end  
return false
```

Interlocks with Forwarding

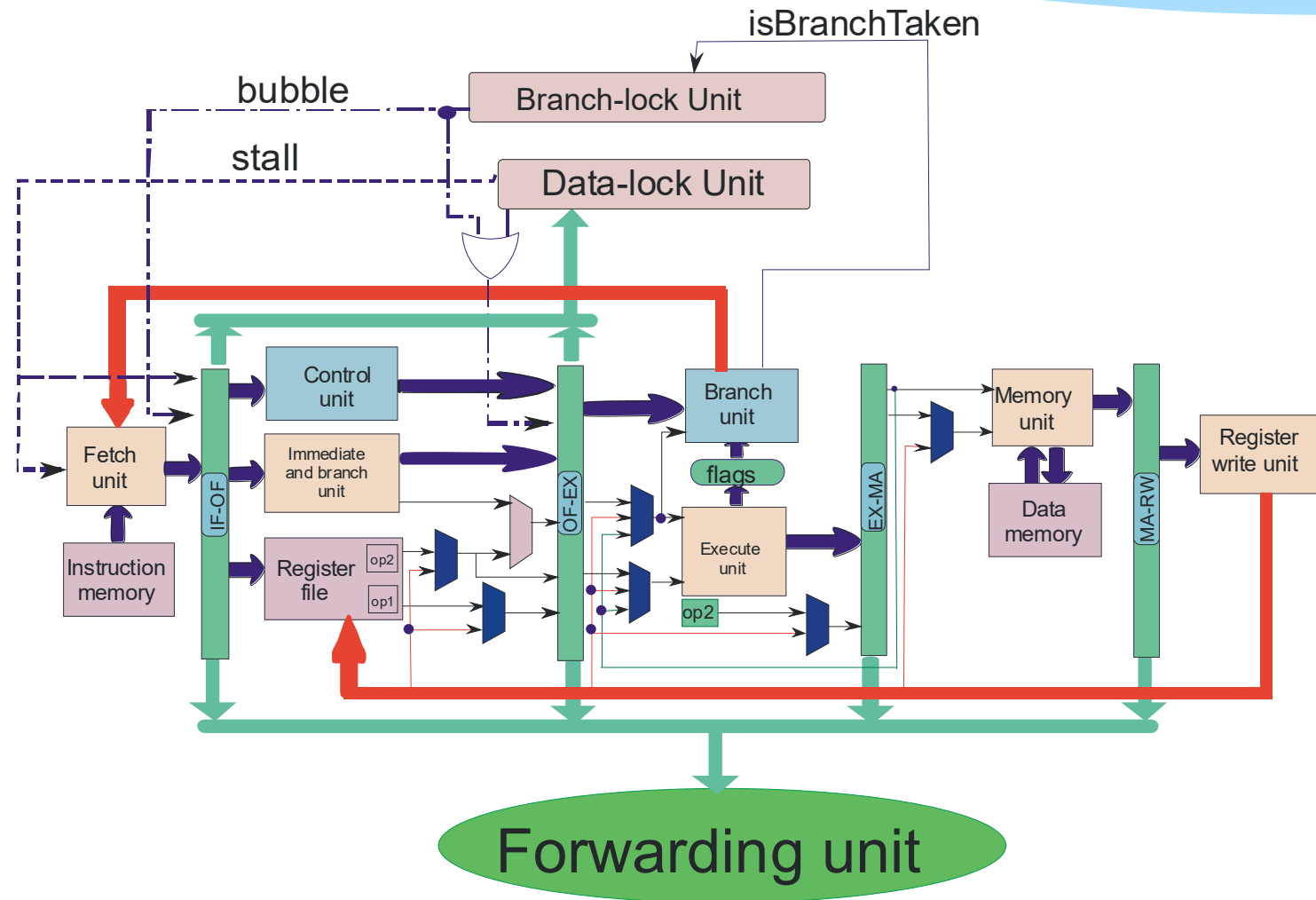
- * Data-Lock

- * We need to only check for the load-use hazard
- * If the instruction in the EX stage is a load, and the instruction in the OF stage uses its loaded value, then stall for 1 cycle

- * Branch-Lock

- * Remains the same as before.

Complete Data Path



Outline

- * Overview of Pipelining
- * A Pipelined Data Path
- * Pipeline Hazards
- * Pipeline with Interlocks
- * Forwarding
- * Performance Metrics
- * Interrupts/ Exceptions



Measuring Performance

- * What do we mean by the **performance** of a processor ?
 - * **ANSWER** : Almost nothing
- * What should we ask instead ?
 - * What is the **performance** with respect to a given program or a set of programs ?
 - * **Performance** is **inversely proportional** to the time it takes to **execute** a **program**

Compute time of a program

$$\tau = \# \text{ Sec} \times \frac{\# \text{ cycles}}{\# \text{ inst}^n} \times \# \text{ inst}^n$$

$$= \left(\frac{\# \text{ Sec}}{\# \text{ cycles}} \right) \times \frac{\# \text{ cycles}}{\# \text{ inst}^n} \times \# \text{ inst}^n$$

$$= \frac{1}{f} \times \text{CPI} \times \text{inst}^n$$

$$\tau = \frac{\text{CPI} \times \# \text{ inst}^n}{f}$$

$$P = \frac{1}{\tau} = \frac{f}{\text{CPI} \times \text{inst}^n}$$

$$P = \frac{f \times \text{IPC}}{\# \text{ inst}^n}$$

$$\text{CPI} = \frac{1}{\text{IPC}}$$

↓
instⁿ per cyle

arbitrary unit
A.U.

Computing the Time a Program Takes

$$\begin{aligned}\tau &= \#seconds \\ &= \frac{\#seconds}{\#cycles} * \frac{\#cycles}{\#instructions} * (\#instructions) \\ &= \underbrace{\frac{\#seconds}{\#cycles}}_{1/f} + \underbrace{\frac{\#cycles}{\#instructions}}_{CPI} * (\#instructions) \\ &= \frac{CPI * \#insts}{f}\end{aligned}$$

- * **CPI** → **Cycles** per instruction
- * **f** → **frequency** (cycles per second)

The Performance Equation

~~$\alpha = \frac{\text{cycles}}{n * n}$~~
int $\text{save}(n)$ {
 $\text{ret}(n * n)$;
}

$$P \propto \frac{IPC * f}{\#insts}$$

- * **IPC** $\rightarrow$ 1/CPI (Instructions per Cycle)
- * What are the **units** of performance ?
 - * **ANSWER** : arbitrary

Number of Instructions (#insts)

Static Instruction: The **binary** or executable of a **program**, contains a list of **static instructions**.

Dynamic Instruction: A **dynamic instruction** is a running instance of a static instruction, which is created by the **processor** when an instruction enters the pipeline.

- * Note that these are **dynamic** instructions
 - * **NOT** static instructions
- * A smart compiler can reduce the number of executed instructions

Number of Instructions(#insts) – 2

- * Dead code removal

- * Often **programmers** write **code** that does not determine the final output
- * This code is **redundant**
- * It can be identified and removed by the **compiler**

- * Function inlining

- * Very small **functions** have a lot of **overhead** → call, ret instructions, register spilling, and restoring
- * Paste the code of the **callee** in the code of the **caller** (known as **inlining**)